# Transport Unit
# 设计文档

介绍产品运行模型的层级结构、数据归属与坐标计算，
说明对象的建立、刷新、跨工位流转及异常处理。

# 目录

**1 设计概述 4**
1.1 一份产品怎样被软件识别 4
1.2 设计目标与职责边界 4
1.3 数据归属与设计约束 5
**2 模块组成与职责划分 5**
2.1 配方与运行对象及执行过程 5
2.2 产品模型怎样参与一次加工 6
2.3 配置上下文是一项共同前提 6
**3 对象模型与数据设计 7**
3.1 对象层级与数据归属 7
3.2 常规模板与异形节点 8
3.3 对象地址与加工顺序 8
3.4 位置结果与过程记录 9
3.5 工艺参与与完成判断 10
3.6 关键字段的来源与有效期 11
3.7 镜像标记与动态配置的不同寿命 12
**4 坐标体系设计 12**
4.1 位置按相对关系保存 12
4.2 单层变换与逐层合成 12
4.3 一个焊点位置的完整计算 13
4.4 工位偏移与分段位置 14
4.5 定位和测高如何影响坐标 15
**5 对象构建与初始化设计 15**
5.1 创建一棵可使用的运行树 15
5.2 常规布局的展开与配置整理 16
5.3 异形节点的展开 16
5.4 工艺状态与工序键的装配 17
5.5 异形配置节点的编辑与运行树的重建 17
5.6 配方 Mapping 与生产 Mapping 的职责 18
**6 生命周期与生产协作 20**
6.1 正常生产链中的三个有效期 20
6.2 启动与 S1 生产入口 20
6.3 从点胶段交给固晶段 21
6.4 S2 为什么通常保留 S1 传来的对象 22
6.5 替代路径会在哪里改变连续性 23

6.6 物料搜索与人工补料怎样改变 TU ..... 24
6.7 搜索与人工确认的完整对象契约 ..... 24
6.8 分段点胶从消息划分到坐标回移 ..... 25
6.9 其他输送入口如何处理对象 ..... 27
6.10 主动清空载具与局部清空 ..... 27
6.11 辅助对象与补偿工作模式 ..... 28
**7 主要接口与使用约定 ..... 30**
7.1 创建和刷新操作的选择 ..... 30
7.2 产品地址查询 ..... 30
7.3 步骤记录与动作记录 ..... 32
7.4 坐标与状态接口 ..... 32
7.5 步骤与转换接口的调用契约 ..... 32
7.6 异形节点反查与位置辅助接口 ..... 33
**8 一致性约束及失败恢复边界 ..... 34**
8.1 配方与对象树及步骤的一致性 ..... 34
8.2 交接成功与交接中断 ..... 35
8.3 人工操作改变哪些记录 ..... 35
8.4 保存的范围决定可恢复的范围 ..... 36
8.5 Mapping 导入重建的是哪一份产品 ..... 36
8.6 交接在不同位置中断后的可见状态 ..... 37
8.7 角度示教的写回次序与取消边界 ..... 38
**9 贯穿实例与设计核对 ..... 40**
9.1 固定一组完整输入 ..... 40
9.2 从 S1 创建到 S2 处理的连续快照 ..... 40
9.3 异形产品的连续对象与坐标例子 ..... 41
9.4 用条件变化检查设计理解 ..... 42
**附录 A 源码依据与阅读定位 ..... 43**
**附录 B 术语与图表索引 ..... 45**
B.1 常用术语 ..... 45
B.2 图目录 ..... 46
B.3 表目录 ..... 46
**附录 C 当前实现的特定边界 ..... 47**

# 1 设计概述

## 1.1 一份产品怎样被软件识别

设备上的产品具有层级。一份载具产品中可以有多块基板，每块基板中有若干基岛，每个基岛上又有多个加工位置。点胶时说“加工 A 点”并不足以确定对象，因为不同基岛上都可能存在名为 A 的焊点。软件需要同时知道它属于哪份产品、哪块基板和哪个基岛，才能读取正确的位置与加工记录。

Transport Unit，简称 TU，承担这项组织工作。它在软件中表示一份正在流转的产品，以自身为根，把基板、基岛和焊点组成一棵运行对象树。树中的每个节点有自己的位置与运行信息；具体工序的完成记录保存在焊点上。输送、定位、测高、点胶和固晶从这棵树取得各自所需的数据。这里的 TU 是产品模型；物理载具如何被皮带传送、气缸如何动作，由输送控制模块完成。 [1] [8]

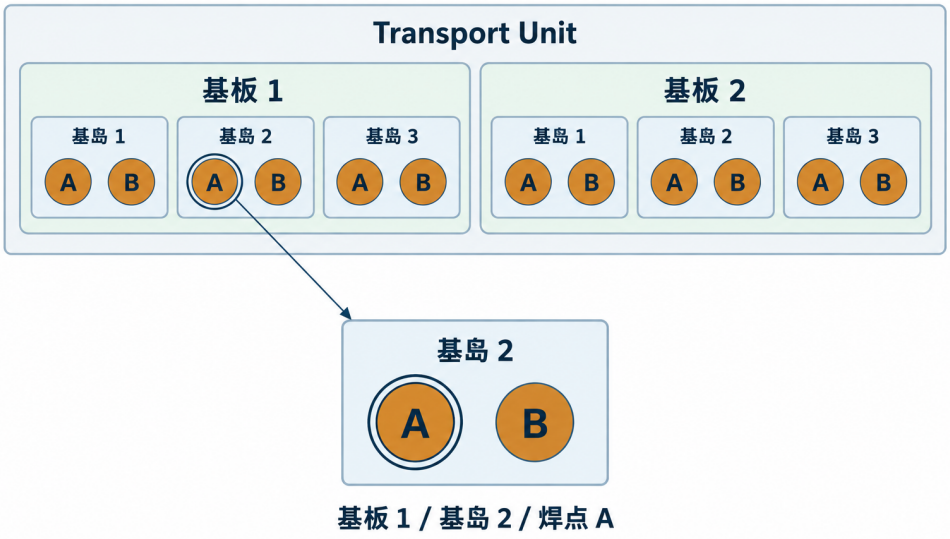



图 1-1 产品层级与一个具体焊点的地址

图中有 2 块基板，每块含 3 个基岛，每个基岛有 A、B 两个焊点。圈出的对象为“基板 1 / 基岛 2 / 焊点 A”。下文用这一说明性产品贯穿对象、坐标和状态的解释，图形不表示设备实物尺寸。

产品层级同时给出了两种关系：集合回答“这个对象属于谁”，父坐标回答“这个位置相对于谁”。两者在创建时一起建立，运行时却可以有不同的变化。比如，跨工位交接后仍使用原来的焊点对象，但它的坐标对象已经被重新建立。理解这两种关系，是读懂 TU 设计的起点。

## 1.2 设计目标与职责边界

TU 首先要让各业务模块使用同一套产品地址。基板和基岛的序号、焊点名称与工序配置共同决定当前加工对象，业务不必分别再组织一套产品层级。其次，位置按父子关系表达，整

体位姿和局部位姿可以在不同层更新。最后，加工记录附着在运行对象上，使同一份产品的工序进度可以在满足条件的交接路径中延续。

这三项目标各有对应的边界。TU 读取当前配方，并没有把创建时的全部配置冻结为私有副本；它保存数据，也没有自行编排或驱动全部动作；它提供完成属性，也不能用一个“完成”字段代替区段有料、工艺参与、处理结果和工序进度。设计评审需要分别检查这些含义。

本文面向软件研发、技术评审以及设备调试人员。前文先建立产品与数据概念，随后解释坐标和构建过程，再沿正常生产链说明对象如何被使用。接口和异常章节用于查阅，最后的贯穿实例可用于核对理解。定位结果怎样影响产品坐标属于本篇范围；图像识别、芯片纠偏和最终贴装补偿算法的完整设计另册说明。

### 1.3 数据归属与设计约束

当前实现分别保存产品身份、几何和进度。正常交接可沿用运行对象、重建下一阶段坐标，并保留各焊点已完成的工序。这是对构建、刷新与交接行为的归纳；不同入口是否保留对象，要看其实际条件。 [1] [4] [8]

这种组织方式依赖三项约束：对象树要与当前配方和工序定义匹配；地址要按所调用接口规定的 Index 或 WorkIndex 解释；当前坐标要从当前产品对象取得。破坏其中任一项，即使引用不为空，动作也可能读到与本次产品或本次消息不一致的数据。第 7 章把这些约束落实为接口契约。

研发评审应检查数据归属和调用前提；设备调试应对应同一产品、同一时刻查看区段状态、对象引用、坐标和焊点进度。两类读者使用的是同一套设计事实，关注的观察入口不同。

## 2 模块组成与职责划分

### 2.1 配方与运行对象及执行过程

同一份产品涉及三类不同的信息。配方描述产品应当怎样组织和加工，例如基板数量、示教位置、焊点名称与工序选择。运行对象记录这一份产品目前的情况，例如当前坐标、定位结果和已完成的工序。执行过程则描述任务接下来选择什么动作，以及动作何时允许执行。三类信息相互协作，但没有被合并成一个对象。

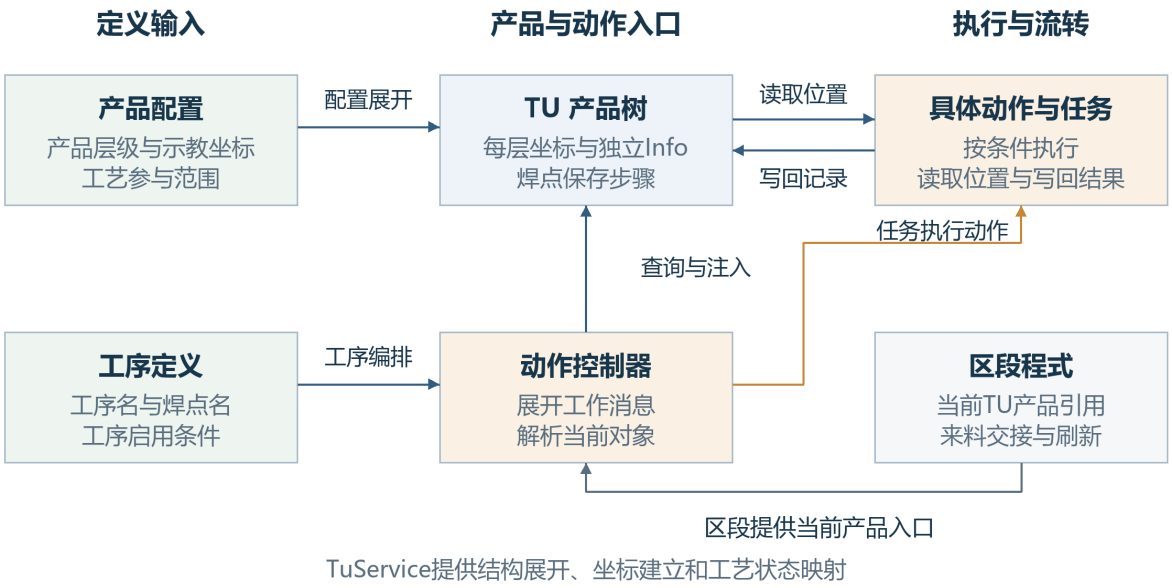



图 2-1 产品配置 运行模型与动作执行的分工

实线箭头标出配置展开、对象查找与结果写回。区段程式持有当前 TU；动作控制器持有工作消息；具体动作读写当前产品。图中的协作关系不代表机械动作必须按图上的空间方向执行。

ProductConfiguration 是配置入口。TransportUnit、Substrate、Module 和 BondPosition 组成运行树。TuService 提供坐标创建、索引换算和工艺状态映射等辅助操作。S1、S2 动作控制器把工序定义展开为工作消息；点胶任务与固晶任务循环取出消息对应的动作，完成条件判断并执行。输送模块负责来料、交接与放行信号，以及区段中的 TU 引用。 [1] [2] [3] [6] [7] [8]

## 2.2 产品模型怎样参与一次加工

一次焊点动作需要把工作消息解析回产品对象。以点胶侧为例，动作控制器先从点胶段取得当前 TU，再按工作序号找到基板和基岛，最后通过当前工序中的焊点名称取得 BondPosition。点胶动作由这个对象得到位置、参与范围和步骤记录，而不是仅凭动作名称决定坐标。固晶侧在初始化当前消息时也沿相同层级解析对象，再供后续动作使用。 [3]

这种分工使动作编排和产品记录可以分别维护。任务为下一份产品复位工作消息时，不必清除上一份 TU 上的工序字典；输送交接替换区段引用时，后续动作又可以从新的当前 TU 解析对象。代价是调用方必须使用一致的配方、工序与产品地址，并区分当前对象和先前缓存的引用。

## 2.3 配置上下文是一项共同前提

常规模式下，BaseMatter.Config 通过当前 ProductConfiguration 动态取得配置：TU、基板和基岛按类型读取，焊点再按名称查找单点配置。异形模式下，运行对象通过关联节点取得配置。因而 TU 对象的存在，并不能证明它仍对应当前生效的产品定义。 [1] [2]

例如，两个工位中同时存在不同运行 TU 时，它们的常规 Config 读取仍指向当前配方。改变基板数量或焊点名称后，旧树不会在下一次读取 Config 时自动变成新结构。TU 类也没有通过 RecipeName 字段完成配置版本隔离。设计使用的前提应明确为“当前对象树、配置及工序定义相互匹配”；不能把一个名称字段当成已经生效的版本约束。

纯几何修改与结构修改需要采用不同操作。前者可以在结构仍一致时刷新坐标，后者需要重新构建产品树。这个区别贯穿后面的初始化、生命周期和接口约定。

# 3 对象模型与数据设计

## 3.1 对象层级与数据归属

TU 保存基板集合，基板保存基岛集合，基岛保存焊点集合。每个对象只直接管理自己的下层，整份产品的遍历和汇总通过逐层展开完成。图 1-1 的例子包含 1 个 TU、2 块基板、6 个基岛和 12 个焊点，共 21 个运行对象。GetAllBaseMatter 只返回根的后代，因此在这个例子中得到 20 个对象。清定位、清测高等整树操作先处理根，再处理这些后代。 [1]

四类对象继承 BaseMatter，共用名称、对象类型、序号、配置访问、运行信息、坐标和状态等能力。各层的运行信息具有不同的具体类型：TU 使用 TransportUnitInfo，焊点使用 BondPositionInfo。基板与基岛负责组织下层；焊点则保存 S1、S2 工序字典，是具体工序完成情况的记录单位。

表 3-1 主要对象的职责与直接数据

| 对象 | 直接包含 | 本层主要作用 |
| --- | --- | --- |
| TU | 基板集合 | 一份产品的入口；承载根坐标及产品级信息 |
| 基板 | 基岛集合 | 表示基板局部位置；汇总下层工艺和完成情况 |
| 基岛 | 焊点集合 | 表示岛局部位置；组织具体加工位置 |
| 焊点 | 本身不再展开产品子层 | 记录单点位置、工序进度及加工过程信息 |

同名焊点的运行数据彼此独立。6 个岛中都可以有 A 点，它们读取相同的常规 A 点配置，但分别拥有自己的 Info 与工序字典。一个 A 点加工完成，不会把其他岛中 A 点的工序同时置为完成。配置复用与记录独立是本模型的一项基本关系。

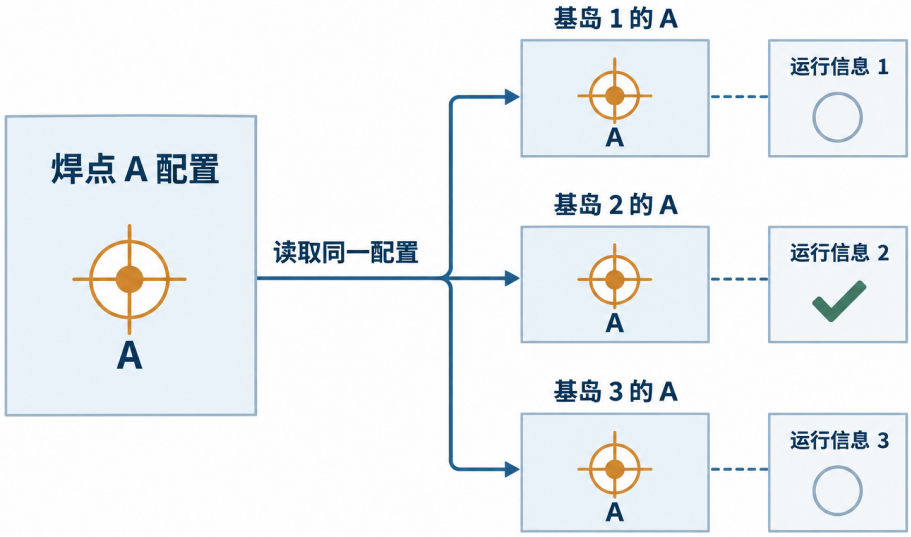



图 3-1 重复焊点共享配置定义而分别保存运行记录

三个 A 点来自不同岛。它们的配置定义相同，运行信息与完成记录分别归属各自对象。图中空心圆与勾仅用于区分未完成和已完成记录，不表示加工比例。

## 3.2 常规模板与异形节点

常规产品采用模板重复展开：使用一套基板配置创建多块基板，再使用一套基岛配置为每块基板创建基岛，最后根据焊点配置列表为每个基岛建立焊点。数量、阵列坐标和工作方向集中配置，适合用较少参数表达规则布局。运行树中的实例仍各自独立。

异形产品使用 OppositeSex 节点树。TU 根节点的 DownConfigs 对应基板，基板的下层对应基岛，基岛的下层对应焊点。每个运行对象保留所关联的节点，节点分别提供名称、位置和配置名称。分支的实际数量取决于节点树，不能由常规基板或基岛的全局 Count 推断。 [1] [2]

异形节点的配置也有共享关系。节点 Config 按 ConfigName 从公共配置列表查找；未找到时，当前实现会创建默认配置、加入列表并保存产品配置。因此“每个节点关联配置”并不意味着每个节点总有一份不可共享的配置，更不意味着读取 Config 一定没有副作用。几何节点与工艺配置名称应在设计中分别理解。

## 3.3 对象地址与加工顺序

Index 表示当前树中的对象序号。常规创建从 1 开始，并以 Index 减 1 读取坐标表。WorkIndex 表示加工编排使用的工作序号，由行列、排列方式与加工方向计算。普通顺序下两者可以相同，蛇形或其他方向下就可能不同。当前动作的基板与基岛查询使用 WorkIndex，因此它不是一个只影响界面显示的附加字段。 [3] [5]

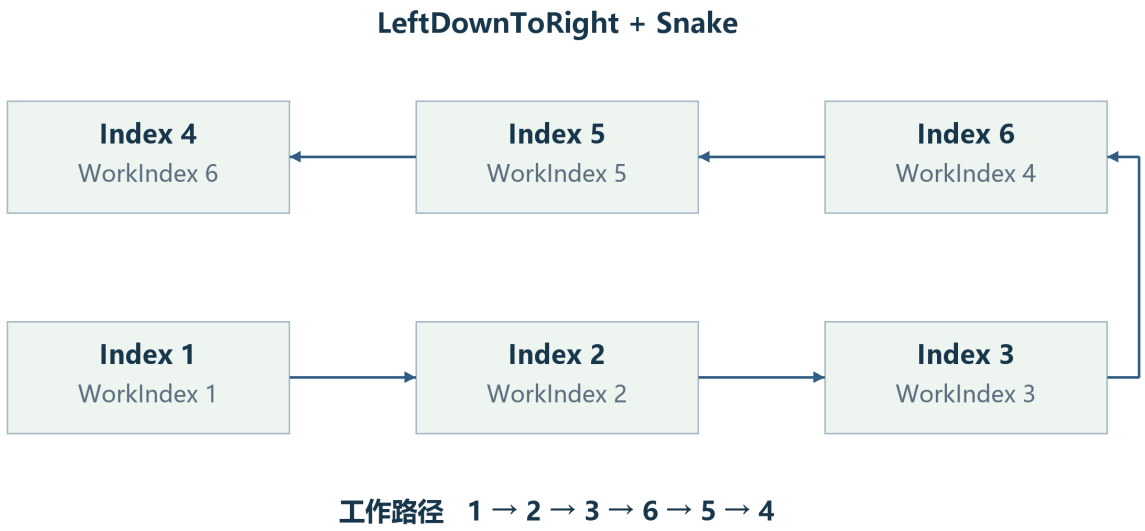



图 3-2 一组两行三列阵列中的对象序号和工作序号

本图另取两行三列阵列说明 LeftDownToRight 与 Snake 组合，不改变贯穿实例的产品数量。各对象 Index 为 1 至 6，WorkIndex 为 1、2、3、6、5、4，所以工作路径按对象序号读为 1→2→3→6→5→4。

列表下标也需要与这两个序号区分。S2 的大数量路径会按 WorkIndex 原地排序集合；Refresh 又会沿现有集合重新赋 Index 并计算常规 WorkIndex。行列索引没有在所有刷新路径中完整重建。对运行过程的持续追踪，应同时明确 TU 实例、当前阶段和层级地址，不能把“列表第几个元素”当作永久不变的产品身份。

## 3.4 位置结果与过程记录

先区分三种存放位置：配置回答这一类点应怎样加工；坐标回答当前这一点相对父层的位置；Info 回答这一点已经取得哪些结果、完成哪些工序。图 3-3 将这三种归属展开到两个同名 A 点。

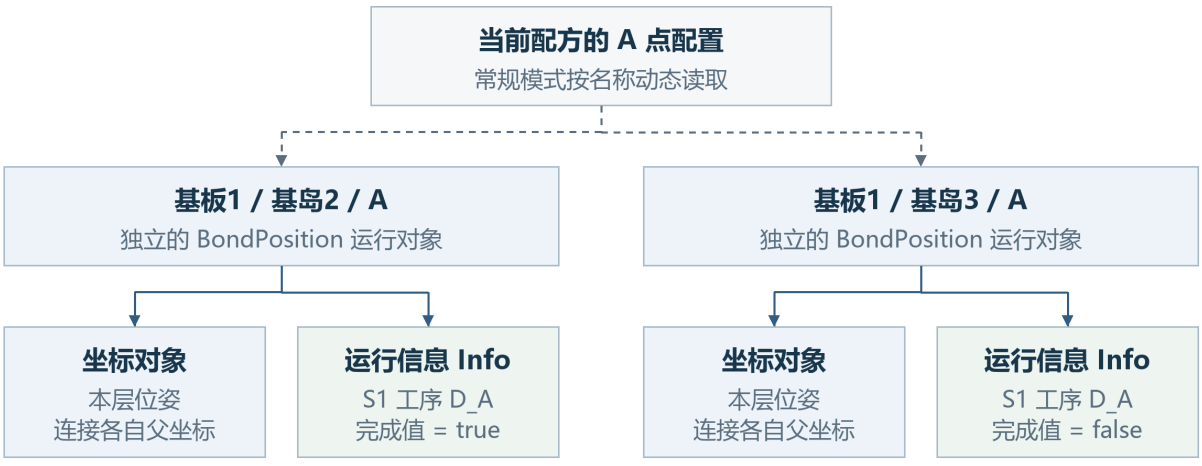



图 3-3 配置读取与运行数据的持有关系

两个 A 点分别位于基岛 2 和基岛 3。灰色框是共享读取的当前配置，蓝色框是各自的对象与坐标，绿色框是各自的运行记录。虚线表示读取配置，实线表示对象持有数据；一个 A 点的 D_A 变为 true，不改变另一个 A 点的记录。

BaseInfo 保存定位结果和测高结果等公共数据。LocateResultInfo 包含 P1 至 P4 的定位信息；VisionResult 分别保存识别结果、对象局部坐标中的结果点、G0 结果点和拍照位置。保存定位信息后，定位流程继续调用坐标更新逻辑，将测得信息转化为当前对象的位姿。结果记录和坐标变化是两个相关步骤，不能只检查其中一个就认为位置更新已经完整完成。 [4] [5] [11]

测高信息的主要数值是实测平均高度与示教平均高度之差。两侧测高流程均把差值写入 MeasureHeightResult，再加到本层坐标的 Z 平移，并设置 IsMeasureHeight。位置转换时，后代通过父链受到这项 Z 变化影响。数值属于哪一层、是否已经作用于坐标，是理解测高记录的关键。

IsVisioned 与 IsMeasureHeight 表示相应流程的处理标记，不能独立解释为真实拍照或测量成功的全部证据。例如定位关闭时，领域方法可以直接把 IsVisioned 置为 true；动作层又可能因配置或完成状态而跳过。Refresh 清除这些标记，但保留旧 Info 数值。标记、结果数值和当前坐标的有效阶段需要一起看。

TU 专属信息中的 IsProduct 用于判断生产入口是否需要重新创建对象。它不是本工位完成标记，也不是质量结果。配方名称、批次、产品号和面别等字段可由信息编辑界面写入，两个有参构造并不自动填满这些内容。焊点专属信息另外保存芯片偏移、实际位置、取片位置和力值等过程量，其使用取决于具体动作。 [4] [9]辅助对象与特殊工作模式的实例归属另见 6.11 节。

## 3.5 工艺参与与完成判断

MatterProductState 表示对象参与哪些工艺：Enable 参与 S1 和 S2，EnableInSystem1 只参与点胶侧，EnableInSystem2 只参与固晶侧，Disable 两侧均不参与。基板与基岛的参与范围由下层汇总：只要存在两侧参与对象，或同时存在仅 S1 和仅 S2 参与对象，上层就覆盖两侧；全部停用或集合为空则汇总为 Disable。TU 没有采用同样的参与范围汇总属性，不能把四层全部画成完全对称的状态模型。 [1]

工序完成情况以焊点字典为依据。字典键是工序名称，false 表示未完成，true 表示已完成。对本侧不参与的焊点，外层完成属性可直接满足；参与时再进入字典检查。基岛要求所有焊点完成，基板要求所有基岛完成，TU 要求所有基板完成。空集合的汇总循环也可返回 true，所以完成属性不能替代产品结构完整性检查。

表 3-2 参与本侧工艺时的完成函数规则

| 条件 | S1 完成函数 | S2 完成函数 |
| --- | --- | --- |
| 本侧字典为空 | true | true |
| S1 非空但 S2 为空 | false，即使 S1 各项已完成 | S2 自身为空，返回 true |
| 两侧字典非空 | 所有 S1 值为 true 时返回 true | 所有 S2 值为 true 时返回 true |

表 3-2 保留了当前实现中两侧不对称的检查条件。EntityState 又是另一维度，取 Process、Success 或 Fail。固晶动作要求当前焊点处于 Process，并同时满足参与范围和工序未完成条件。完成字典不会在这里被自动等同于 EntityState，更不能直接代替实物质量结论。 [1] [4] [7]

## 3.6 关键字段的来源与有效期

字段的有效期取决于它属于配置、当前几何还是运行记录。表 3-3 与表 3-4 把相同操作作用于不同数据的差异放在一起比较。初始值区分新建 Info 与完成工序注入两个阶段；坐标中的长度采用 mm，Degree 采用 rad，布尔值与工序字典没有物理单位。 [1] [3] [4] [5] [11]

表 3-3 配置与位置数据的读写规则

| 关键字段 | 建立时的含义 | 写入与主要使用 | Refresh 后的含义 |
| --- | --- | --- | --- |
| 配置入口<br>Config | 按当前配方或异形节点动态取得 | 构建和动作条件读取配置；不保存为私有配方快照 | 仍按当前上下文读取 |
| 本层坐标<br>CoordinateSystem | 连接父坐标，复制本层配置位置 | 构建、刷新、定位和测高修改；移动动作读取转换结果 | 替换坐标实例，重新取配置几何 |
| 定位结果与标记<br>LocateResultInfo<br>IsVisioned | 新 Info 建立结果容器，标记为 false | 视觉流程保存结果并更新坐标；动作条件读取标记 | 旧结果保留，标记清 false |
| 测高差与标记<br>MeasureHeightResult<br>IsMeasureHeight | 新 Info 为 0mm / false | 测高流程写高度差并增量改 Z；后代转换使用父层 Z | 差值保留，标记清 false；旧增量不再包含于新坐标 |

表 3-4 入口标记与过程记录的读写规则

| 关键字段 | 建立时的值 | 写入与主要使用 | Refresh 后的含义 |
| --- | --- | --- | --- |
| 生产入口标记<br>IsProduct | TU Info 中初始为 false | S1 / S2 入口及 Mapping 可置 true；生产入口据此判断重建 | 保留；不能当作工序完成证据 |
| 工位偏移标记<br>IsSystem1Offset<br>IsSystem2Offset | TU Info 中两者均为 false | 本侧偏移方法置 true 并避免本侧重复累加 | 均清 false；自身不重新应用偏移 |
| 焊点工序字典<br>S1RemainingSteps<br>S2RemainingSteps | 焊点新 Info 中为空；注入缺键后值为 false | 动作成功写 true；参与完成判断；人工操作可修改 | 键和值均保留 |
| 身份识别记录<br>IsIdentity / IdentityCode | 各层新 Info 为 false / null | 身份识别保存代码与标记；识别动作读取标记；离线可只设标记 | 标记与代码均保留 |

控制器消息的 Finished 属于动作编排。新消息默认 false，任务处理后标记已完成；控制器 Finish 复位工作消息，TU 的 Refresh 不复位消息。它与本表中的产品运行记录分别维护，详见 6.4 节。 [3]

身份识别标记与定位标记的寿命不同。以两侧基板识别动作为例，IsIdentity 已经为 true 时会跳过；Refresh 只清定位和测高标记，因此交接不会自动要求重新读码。离线分支还能只置识别标记而不产生代码，所以“标记为 true”和“保存了有效 IdentityCode”需要分别判断。两侧物料识别的忽略分支执行本层停用操作后返回，不经过正常的代码保存段；上层对象按其传播规则影响下层。 [13]

EntityState 也不属于这些完成标记。它在 BaseMatter 中初始为 Process，动作读取它决定是否进入处理；把某工序字典值置为 true 并不会同时把它改成 Success。对当前产品的判断应同时明确所属对象、字段及阶段。

### 3.7 镜像标记与动态配置的不同寿命

焊点的 IsMirror 在构造时从常规焊点配置或异形节点配置复制。它是运行对象上的独立布尔值；BondPosition.Refresh 只重建坐标，没有再次赋值 IsMirror。修改配置中的镜像选项后，旧焊点的镜像标记可能仍保留创建时的值。普通上视纠偏读取的是运行焊点 IsMirror，因而这项变化要按重新构建对象处理，不能把刷新坐标理解为刷新全部配置派生字段。 [1]

## 4 坐标体系设计

### 4.1 位置按相对关系保存

每层产品对象持有一个 GeneralCoordinateSystem。Distance 给出相对直接父坐标的三维平移，Degree 给出绕 Z 的平面角度，角度计算使用弧度。焊点相对基岛，基岛相对基板，基板相对 TU；TU 的父坐标从机器配置中按名称 TransportCoordinateSystem 取得。再沿机器父链，产品点可以转换到 G0 全局参考。 [1] [5]

采用相对位置后，整体变化可以留在上层表达。TU 整体平移时，各焊点仍保存自己的局部位置；下一次转换会使用当前父链。接口调用必须区分产品局部点、G0 点和运动轴目标，同名 XYZ 字段不能直接互加。

### 4.2 单层变换与逐层合成

以下采用普通平面刚体模型解释逐层合成。输入局部点为 x、y、z，本层平移为 dx、dy、dz，角为 θ；先旋转 XY，再叠加平移，Z 只作平移。下式给出父参考系位置；实际转换由本目录调用的坐标接口完成。

$$X_p = d_x + x\cos\theta - y\sin\theta \tag{4-1}$$

$$Y_p = d_y + x\sin\theta + y\cos\theta \tag{4-2}$$

$$Z_p = d_z + z \tag{4-3}$$

本目录的动作通过 SelfPosToG0 把产品局部点送入机器全局参考，通过 G0PosToSelf 把测得结果带回当前对象。DegreeInG0 的调用结果在需要轴角时乘 180/π，进入芯片旋转公式后再按接口要求使用度或弧度。下面用普通平面刚体关系解释父子坐标；机器坐标中的相机标定仍作为转换接口的运行前提。

## 4.3 一个焊点位置的完整计算

仍取基板 1、基岛 2、焊点 A。为使每一步可复算，规定 Transport 到 G0 为零平移、零角度；TU 平移为(100,200,0)，角度为 π/2。基板 1 相对 TU 为(20,0,0)，基岛 2 相对基板为(10,5,0)，焊点 A 相对基岛为(10,5,0)，下层角度均为零。所有长度统一按 mm 表达。

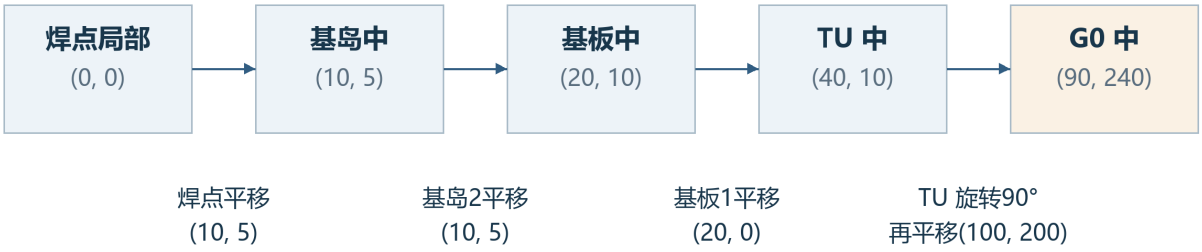



图 4-1 焊点局部原点逐层转换的中间结果

从焊点自己的原点出发，先到基岛得到(10,5)，再到基板得到(20,10)，到 TU 得到(40,10)。TU 旋转 90°并叠加(100,200)后，G0 结果为(90,240)。图中省略恒为零的 Z 值。

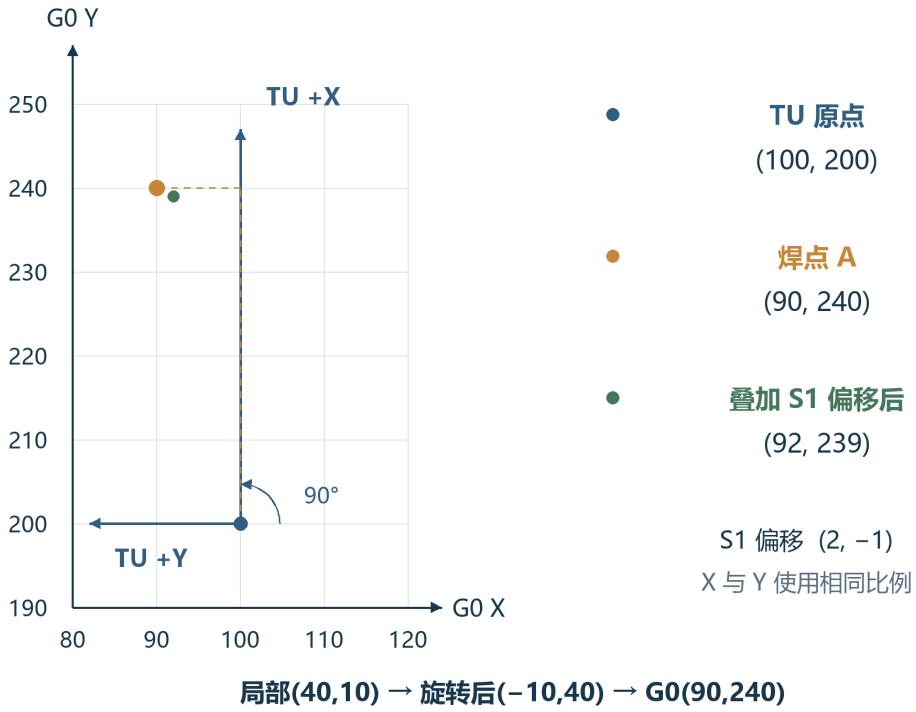



图 4-2 TU 旋转后的局部轴与焊点全局位置

图示为 G0 的局部视窗，刻度直接使用 G0 坐标。TU 局部正 X 指向全局正 Y，局部正 Y 指向全局负 X；所以局部(40,10)在旋转后变成(−10,40)。橙点为叠加 TU 平移后的结果。

算例也说明“相对谁”为什么不可省略。局部 X 增加 40 并不意味着全局 X 增加 40，因为 TU 旋转了 90°。逆算时，(90,240)先减去 TU 平移得到(−10,40)，逆转 90°得到(40,10)，再去掉基板、基岛与焊点平移，回到焊点原点。

## 4.4 工位偏移与分段位置

S1 和 S2 工位偏移直接加到 TU 的 Distance 上，因而影响整棵后代的转换。两套偏移各有一个独立布尔标记：相应标记为 false 时累加并置 true，已经为 true 则返回。该保护只避免同一套偏移被重复累加，并不阻止 S1 和 S2 偏移连续相加。字符串构造不自动选工位偏移；系统参数构造在 Init 后选择对应偏移。 [1]

把算例中的 S1 偏移设为(2,−1,0)，在尚未应用时调用一次后，TU 平移变为(102,199,0)，焊点 G0 位置变为(92,239,0)。再次调用 S1 偏移方法，结果保持不变。Refresh 会重新按配置建立坐标，并把两套标记清为 false，它自身不重新应用任何工位偏移。定位扫描、示教及生产创建应分别按其实际入口判断。

分段点胶的平移与工位偏移使用不同机制。System1MoveDistance 仅在 SubstrateProcessing 为 Divide 时给 TU 的 Distance.X 加 DistanceToRight，回移方法做减法，没有同一套布尔标记保护。具体前后段由配置与区段位置共同决定，不能从“分段”一词推断固定一半数量，也不能把枚举中的 DivideWithInverted 自动视为同一分支。 [1] [2] [9]

### 4.5 定位和测高如何影响坐标

定位先保存测得的结果点，再由 TuService.UpdateEntity 计算用于更新的点和角度，并调用当前坐标的 UpdateCoordinateSystem。被更新层的后代不需要逐点重写；它们下一次向 G0 转换时就会受到父层位姿变化影响。本篇说明数据和几何的归属，定位点选择与角度计算的完整算法另册展开。 [5] [11]

测高则对本层 Z 平移作增量修正。若基板 1 的测高差为+0.12mm，该值写入基板 Info 并加到基板坐标 Z，其下所有焊点转换时都会带上这项高度。Refresh 以后，基板坐标重新来自示教配置，旧差值仍可能留在 Info 中，完成标记已被清除；旧数值存在不代表新坐标已经包含它。

坐标本身是一组可替换的对象。CreateMatterCoordinate 每次新建 GeneralCoordinateSystem、连接父引用，再复制配置点与角度。Refresh 递归执行这一过程，因此保留焊点对象不等于保留它的 CoordinateSystem 实例。需要当前几何时，应从当前产品对象重新取得坐标，而不是继续使用刷新前缓存的句柄。

# 5 对象构建与初始化设计

### 5.1 创建一棵可使用的运行树

有参构造从新的 TransportUnitInfo 开始。Init 先关联异形根配置并建立 TU 坐标，再按常规或异形模式展开下层，随后应用工艺参与配置，最后依次注入 S2 和 S1 工序。Init 返回后生成 TU 时间名称；系统参数构造再加工位偏移。工序注入顺序表示初始化调用先后，不表示设备先执行 S2 再执行 S1。 [1] [2] [3]

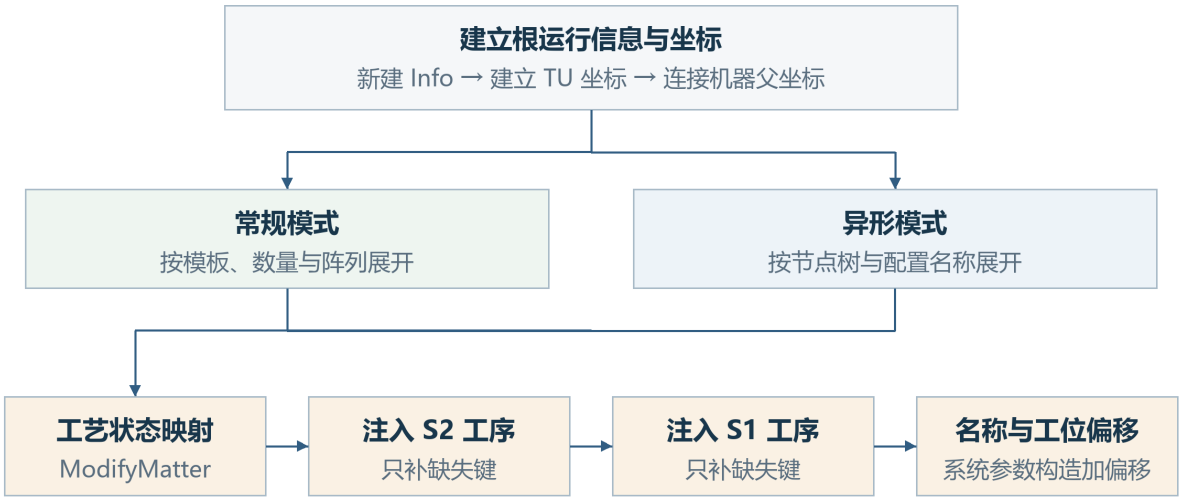



图 5-1 从产品配置到运行对象树的建立顺序

两个结构分支汇合后，才应用工艺参与状态并注入步骤。输出中的坐标、运行 Info 与工序字典分别承担几何、结果和进度职责。示意省略了业务设备动作，因为构建过程本身不执行点胶或固晶。

无参构造用于反序列化，不走这条完整链。普通创建需要当前配方、机器父坐标和两侧工序定义已经可用。IsTransportUnitReady 仅检查 TU 示教结果标记，不能代替数量、坐标表、名称或节点关联的完整一致性检查。

## 5.2 常规布局的展开与配置整理

基板与基岛的常规创建按 Count 循环，每个对象从坐标表的 index−1 位置取几何参数。基岛再按单焊点配置列表创建焊点。矩阵模式建立行列索引，WorkIndex 由排列和工作方向计算。图 1-1 中的 2×3×2 产品由两块基板、每块三个基岛和每岛 A、B 两个焊点展开。

创建还包含配置整理。非多件模式会把数量与行列改为 1，并重建默认坐标；空坐标表会补一个默认点；启用输入间距时，按首点、行列与间距生成阵列坐标，保留首点 Z，新点角度为零。这些操作会改变内存中的配置集合，所以“创建 TU”不是纯只读地复制配方。调用者应把整理后的配置与实际输出一起理解。 [1] [2]

对贯穿实例，基板和基岛都显式设为多件，数量分别为 2 和 3，坐标表长度与数量一致。实例直接给出各层坐标，不依赖未声明的间距自动生成条件。这样后面的对象数量和坐标计算由同一组条件得到。

## 5.3 异形节点的展开

异形创建以 DownConfigs 的实际节点数为准。每层构造把节点关联到运行对象，再建立坐标和自己的 Info；基板、基岛的初始 WorkIndex 等于节点序号。配置名决定读取哪份工艺配置，节点位置决定在父层中的布局，两者不必一一对应。

这条路径使不同基板可以具有不同的下层结构，但也要求节点层级和类型含义正确。常规全局 Count 仍存在于其他便捷接口中，不应据此替代实际异形集合。需要获取真实产品规模时，应沿运行树或相应节点树统计。




图 5-2 常规模板与异形节点的展开差异

左右是两组独立说明数据。常规例中同一基岛模板重复到每块基板，形成相同数量；异形例中两块基板的下层分别有 2 个和 3 个节点。A、B 均表示逻辑加工位置，矩形边界不表示基板实物形状。

## 5.4 工艺状态与工序键的装配

TuService.ModifyMatter 把工艺参与配置写到焊点。异形模式取焊点节点状态；常规大数量模式按基板与基岛索引进入状态结构，再按焊点名匹配；普通模式查找“基板序号＋基岛序号＋焊点名”的个别记录。基板、基岛随后由下层状态计算参与范围。 [2]

工序注入采用另一种匹配。动作控制器遍历已启用工序定义，以 BondPositionName 匹配焊点 Name，再把 ProcessStepName 作为该焊点字典的键。不存在的键添加 false，已经存在的键保留原值，旧键不会自动删除。重复注入可以补充新增工序，而不会把已经完成的工序重新改成未完成。 [3] [4]

工序名称还用于从当前工作消息找回工序定义。贯穿实例分别使用 D_A、D_B 表示 A、B 点的 S1 工序，T_A、T_B 表示对应 S2 工序，避免把同一个工序键同时用于两条不同的焊点定义。A 点和 B 点的地址由工序定义中的焊点名解析，工序名与焊点名承担不同职责。

初始化没有覆盖所有失败的统一回滚。特别是 ModifyMatter 内部异常会清理工艺状态配置并调用保存，已写入的部分运行状态不会因此全部重新映射。构建结果是否完整需要按实际调用检查，不能仅凭“没有收到该方法抛出的异常”推断状态装配已经正确。 [2] [10]

## 5.5 异形配置节点的编辑与运行树的重建

异形产品的编辑对象是 OppositeSex 配置节点树。编程页面选择 TransportUnitGeneral 且 IsOppositeSex 为 true 时装入编辑控件；新增、删除、复制和清空修改的是这棵配置树。运行 TU 仍持有创建时关联的节点和后代集合。保存配方或重画编辑视图不会自动重建当前工位的运行树。 [15]

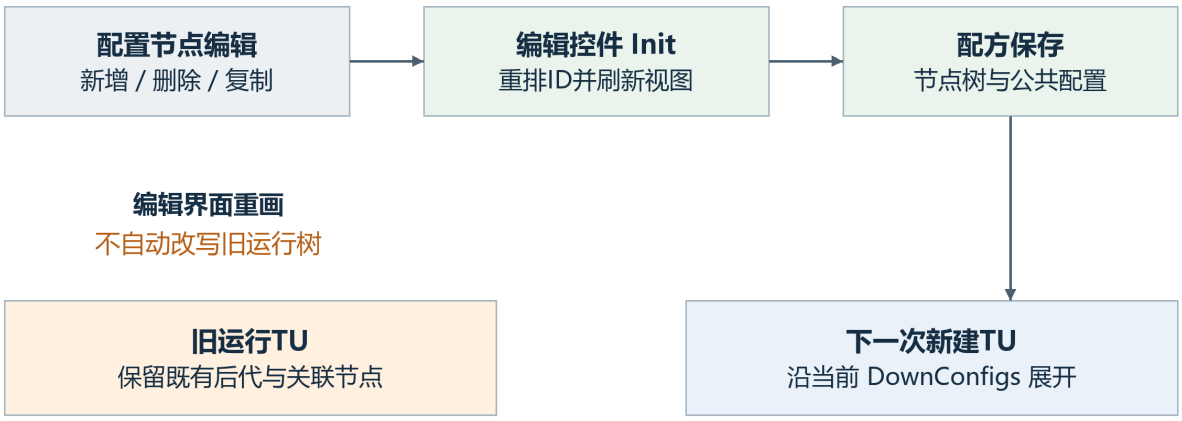



图 5-3 异形配置编辑与运行对象的两个更新阶段

从左到右阅读。上排是配方节点与编辑界面，下排是已有和下一次新建的运行 TU。编辑后的 Init 负责重新整理配置节点 ID 并刷新视图，箭头不表示自动改写旧 TU 后代。新建 TU 才沿当时的 DownConfigs 展开。

新增节点按所选类型加入根、基板或基岛的 DownConfigs，并以输入名称建立 ConfigName。复制窗口按行列循环克隆所选节点，跳过原位，再给复制节点增加列距和行距；调用方修改复制节点的显示名称，未另外改写 ConfigName。因此节点可以有不同位置和名称，同时继续关联同一份工艺配置。克隆内部行为属于目录外接口，本册只采用本目录可见的调用和后续赋值。

删除按节点对象引用从父集合移除。所选节点的 ConfigName 不再被节点树引用时，删除相应公共配置以及该配置名下的四个 Mark PR。清空按钮则清根的 DownConfigs 和 BaseConfigs。编辑控件的 Init 随后重排节点 Id、ParentId 及辅助层级 ID，再刷新树视图；这些 ID 用于配置导航，不能当作产品运行 Index 或永久身份。

对已有 TU，纯坐标变化可在结构及节点关联仍匹配时通过 Refresh 生效；新增、删除或替换配置节点后，要重新建立运行树。旧 TU 仍引用旧节点时，单纯用新节点名称或 ID 不能保证找到对应运行对象。7.6 节给出按节点反查的确切契约，8.1 节给出结构与工序共同一致的前提。

## 5.6 配方 Mapping 与生产 Mapping 的职责

Mapping 既可编辑配方中的工艺参与范围，也可操作正在加工的产品记录。两者使用相似画面，但数据拥有者和保存出口不同。主界面的 TU Mapping 按 UseNewMappingUi 选择新旧配方编辑界面；生产 Mapping 则接收区段程序或真实运行 TU。表 5-1 先确定画面正在操作哪份对象。 [16]

表 5-1 三类 Mapping 入口的数据与保存对象

| 入口 | 持有的 TU | 保存与后续生效 |
| --- | --- | --- |
| 新配方 Mapping | 界面自行构造的 mapping TU | 修改运行样本；关闭且 IsChange 时用 SaveInformation 写回参与配置 |
| 旧配方 Mapping | 界面自行构造的 Mapping 图 TU | ChangeMatterProduct 写参与配置，再构造样本用于重画 |
| 生产 Mapping | 区段当前 TU 或传入的真实 TU | 直接修改本产品参与状态及焊点步骤；不据此保存完整运行快照 |

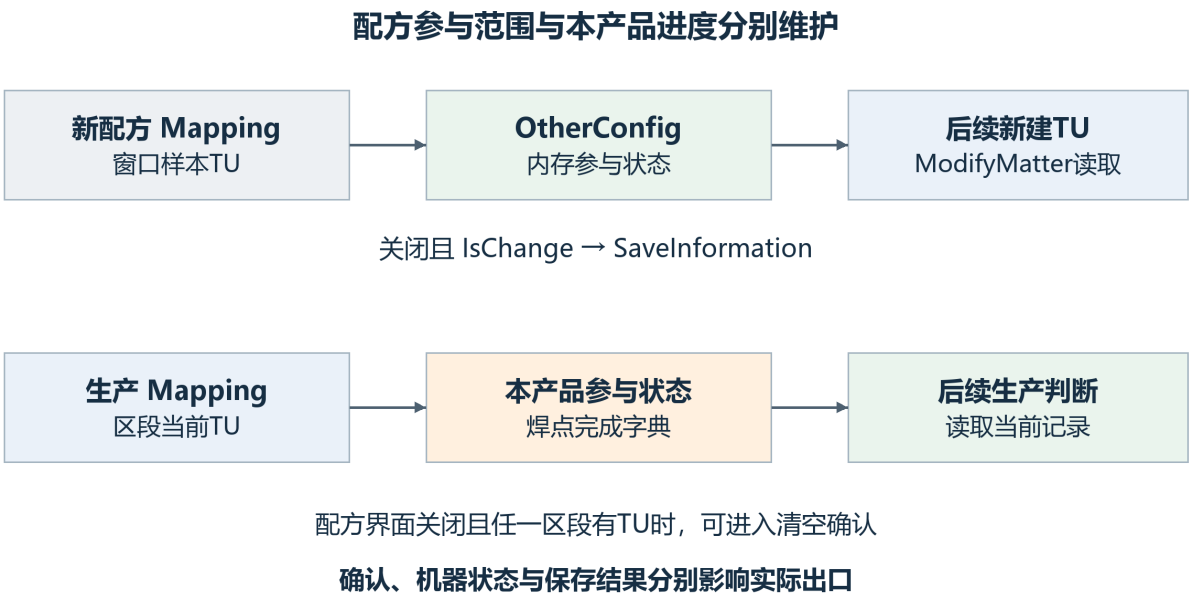



图 5-4 配方 Mapping 与生产 Mapping 的数据去向

上排表示新配方 Mapping：样本 TU 的参与状态写入内存 OtherConfig，再由后续构建的 ModifyMatter 读取。下排表示生产 Mapping 直接修改当前 TU。关闭新窗口时的写回不等于已经保存到文件，两条路径也不保存完整生产快照。

新界面的 SaveInformation 按当前常规 Count 分配参与状态结构，大数量模式记录每个焊点状态，普通模式记录非 Enable 的个别项；它不保存样本 TU 的两侧完成字典。关闭事件及 SaveInformation 本身都未调用配方 Save，参与状态此时只写回内存，不能据此认定已落盘；旧界面关闭时则调用传入的保存回调。写回中若发生索引等异常，方法只弹窗，没有把已清理或写入的配置统一恢复。工作方向和蛇形选项另行写入配方并调用 Save。新界面的清空按钮先清参与配置，再构造新的样本 TU 并保存；不能把它理解为清空生产区段 TU。

主界面关闭任一配方 Mapping 窗口后，若 IsExistTu 检查发现五个区段中任一处仍有 TU，会调用 InitializeTS。这个调用仍要通过工作或暂停状态检查，并由操作者确认后才清除引用，不能写成“关闭 Mapping 必然清空载具”。若保留已有树，也不能据此认定它已经重新执行 ModifyMatter；配置变化与运行对象的生效步骤应分别核对。

# 6 生命周期与生产协作

## 6.1 正常生产链中的三个有效期

产品进入固晶工位时，软件需要继续识别这份产品、计算它现在的位置，并保留已完成的点胶记录。当前设计把这三件事分开处理：正常交接沿用原来的产品对象，重新建立各层坐标，同时保留焊点工序字典。对象、几何和进度因此具有不同的有效期。 [6] [7] [8]

图 6-1 取 S1 加工结束、交接结束、S2 入口三个时刻，跟随同一个 A 点观察变化。S1 入口怎样建立这份对象见 6.2 节；S2 何时会改为新建对象见 6.4、6.5 节。图中的“同一份产品”只适用于图注列出的正常路径。

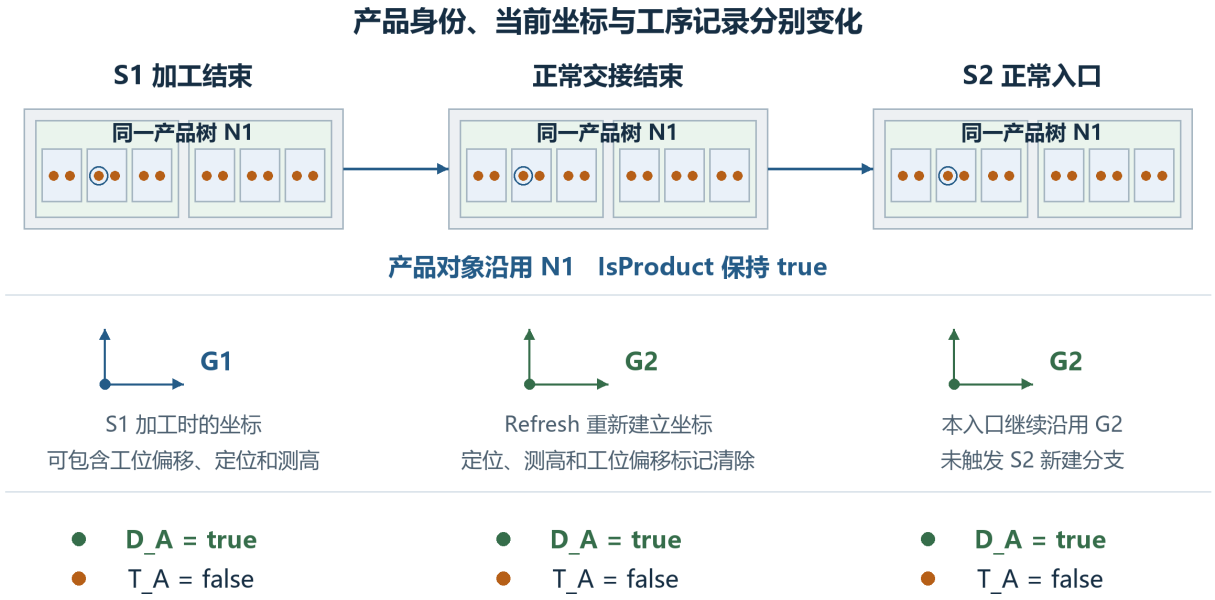



图 6-1 正常 S1 到 S2 路径中对象、几何与进度的有效期

蓝圈标出基板 1、基岛 2 的 A 点。首列取 S1 已完成该点点胶而 S2 尚未加工的时刻；正常自动交接成功，S2 入口保留 IsProduct=true 的对象。三个产品图都表示同一实例 N1。G1、G2 是交接前后两套坐标树的图内代号，不表示实物位移。D_A、T_A 分别是该点的 S1、S2 工序键。新建或异常路径另见 6.5 节及第 8 章。

这一过程需要区分产品模型和动作控制器。TU 中的焊点字典记录产品的工序完成情况；控制器中的工作消息记录本次动作编排走到了哪里。任务结束时重置工作消息，不意味着清空 TU 的加工记录。反过来，新建 TU 会得到新的运行信息，即使控制器仍使用同一套动作定义。

## 6.2 启动与 S1 生产入口

机器启动先准备相关系统，再启动输送任务。满足配置条件时启动点胶任务；固晶侧初始化动作控制器，然后启动固晶任务。各任务随后通过来料与放行信号协作，不能把启动函数中的语句顺序直接解释为完整的实物加工顺序。 [12]

自动点胶任务先建立本轮工作消息，再等待来料。在启用点胶的分支中，如果点胶段当前 TU 的 IsProduct 为 false，任务使用 S1 系统参数重新创建 TU，并把新对象的 IsProduct 设为 true；随后再注入当前 S1 工序。原先由上料或物料搜索形成的 N0 在这里被 N1 替换。N1 的几何从配方建立，并应用 S1 工位偏移。 [6]

点胶循环逐个取动作，检查是否需要执行，再执行并标记当前工作消息。重试返回可以继续本动作；其他非成功结果进入相应停止处理。焊点点胶动作成功后，才把当前工序名对应的 S1 字典值置为 true。控制器标记“消息已完成”与焊点写入“工序已完成”由不同调用承担，不应合并成一个隐含操作。 [3] [6] [11]

## 6.3 从点胶段交给固晶段

交接分为机械移送、软件引用交接和来料通知几个阶段。S1 在未停止时先发出允许移送的信号，再调用控制器 Finish 复位本轮工作消息。输送侧要求点胶段有料、固晶段无料；自动运行还要等到这个放行信号。 [6] [8]

机械移送和检测完成后，输送控制器把点胶段的 TU 引用交给固晶段，然后执行 Refresh。刷新正常返回后，才清空点胶段引用，更新区段状态并保存配置。输送任务取得 Success 以后，再向 S2 发送来料完成信号。图 6-2 按这个先后次序展开。 [8]




图 6-2 自动点胶到固晶交接的主要调用与信号

四条竖向虚线分别表示 S1 任务、输送任务与控制器、区段产品引用、S2 任务。图中把机械移送和检测合并为一个有明确完成条件的步骤。S1 的 Finish 发生在放行之后，与输送动作没有额外的串行等待关系；交接完成信号由输送任务在控制器返回 Success 后发出。 本图为正常未忽略报警的路径，Ignore 后仍继续交接的条件见 8.2 节。

软件交接后，固晶段访问的是 S1 使用过的同一棵产品树。N1 及其基板、基岛、焊点继续存在；交接没有另外创建一份 TU 来接收工序字典。 [8]

Refresh 重新建立各层坐标，清除定位、测高和工位偏移标记；IsProduct 及两侧焊点工序字典保留。设备调试时可能同时看到“点胶已完成”和“需要重新定位”，这两个状态属于不同数据，能够同时成立。表 6-1 再区分区段有料状态、移送状态与产品引用。

输送任务收到控制器成功返回后，发出固晶来料完成信号。固晶任务等待该信号后进入产品处理。放行、实物交接、引用交接和目标来料通知分别位于不同位置，调试时应按这些节点判断停在哪一段，不能只看一个区段中的 TU 是否非空。

区段另外保存有料状态 SubSectionState 与移送状态 SubSectionTransferState。前者的 HasMaterial 和 NoMaterial 用于本次移送的进入判断；后者的 Ready、Transfering 和 Alarm 描述机械移送阶段。它们属于区段程式，与 TU 的 IsProduct 及焊点工序字典分别更新。表 6-1 只描述点胶到固晶这一分支的关键节点。 [8]

表 6-1 点胶到固晶移送中的状态与引用

| 节点 | 区段有料状态 | 移送状态 | TU 引用与数据 |
| --- | --- | --- | --- |
| 允许进入并开始移送 | 源 HasMaterial；目标 NoMaterial | 源先置 Transfering；目标在接料步骤另置 Transfering | 此阶段产品仍由源区段持有 |
| 成功出口 | 源 NoMaterial；目标 HasMaterial | 两侧均置 Ready | 目标持有原 TU 并已 Refresh；源清空；步骤保留 |
| 目标 Refresh 抛异常 | 未保证达到成功出口状态 | 没有覆盖全部状态的统一回滚 | 目标可能已接收源引用；源尚未清空，坐标可能部分重建 |

物料检测失败的相应分支还会把该侧移送状态设为 Alarm，再由实际处理分支决定重试或退出。Alarm、焊点 Fail 与某个工序未完成不是同一个状态；恢复判断需要同时对应区段、产品和动作阶段。

## 6.4 S2 为什么通常保留 S1 传来的对象

固晶任务取得来料后补充 S2 步骤键。启用固晶且输送已配置时，只有当前 TU 的 IsProduct 为 false 才会按 S2 参数重新创建对象并置为 true。正常 S1 已经把 N1 的标记设为 true，交接刷新又保留了该标记，所以这条正常路径不会在 S2 入口无条件新建 TU。S1 的加工记录正是沿这一条件得以延续。 [7]

固晶控制器按当前工作消息解析基板、基岛和焊点，动作再读取它们的当前状态和坐标。以固晶动作而言，是否执行同时涉及工艺参与、EntityState 及本工序是否未完成；成功路径写入当前焊点的 S2 步骤记录。动作全部处理后，控制器 Finish 复位工作消息，再由发送流程决定下一步放行。

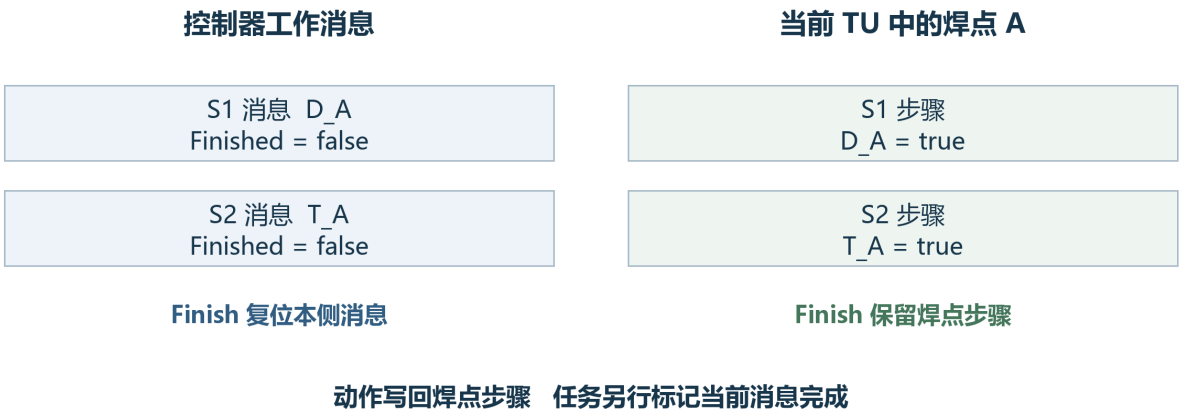



图 6-3 工作消息与产品步骤是两套记录

左侧属于动作控制器，右侧属于当前 TU 的焊点。Finish 复位左侧消息；右侧 D_A、T_A 的值由具体工序写回与明确的人工操作改变。本图只展示 A 点对应的两个工序，其他焊点拥有各自的记录。

启用输送的正常发送路径在未停止时发出允许向待下料段移送的信号。后续成功移送继续采用目标引用接收、刷新坐标与清空源引用的模式，直至产品离开区段。在未配置输送的路径中，发送流程可按配置导出 Mapping，清空固晶段 TU 并置无料，再提示人工取走产品并停止。因此最终对象退出方式也取决于实际生产配置。 [7] [8]

## 6.5 替代路径会在哪里改变连续性

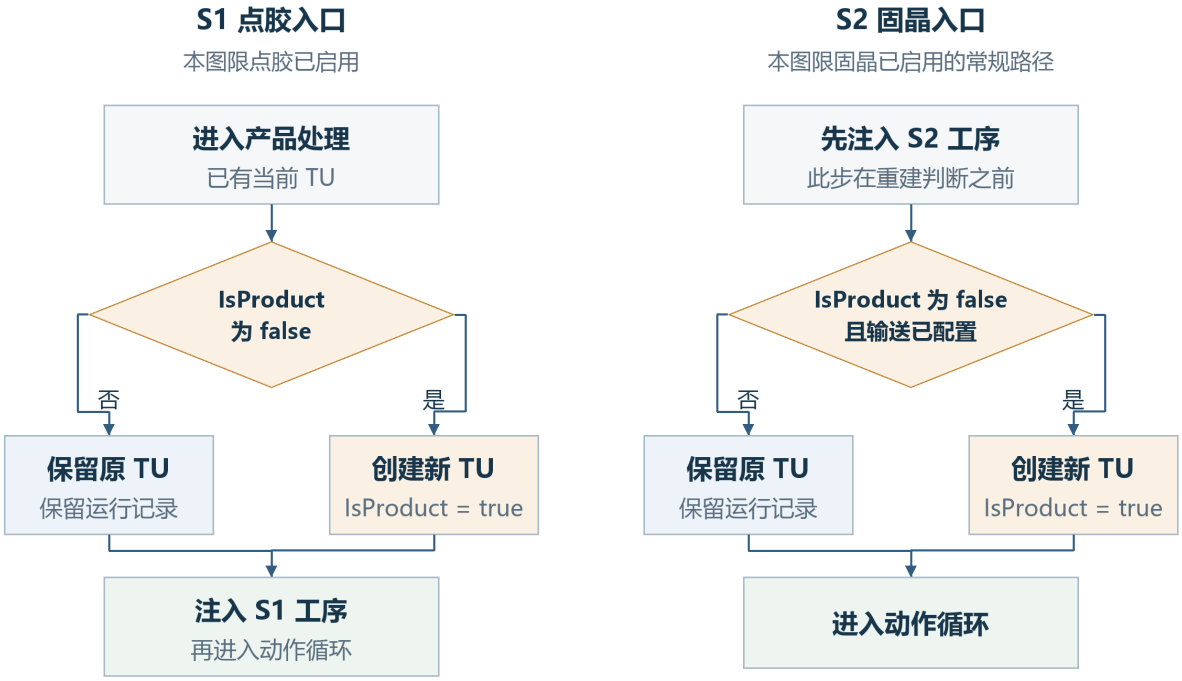



图 6-4 S1 与 S2 生产入口对保留或新建 TU 的判断

图中前提为正常入口已有可用 TU，并分别处于 S1、S2 启用分支；补偿工作模式等特殊入口另行判断。S1 先判断是否新建，再注入本侧工序；S2 先注入工序，随后在启用分支按 IsProduct 与输送配置判断是否新建。图中条件和次序按当前代码分别保留。

点胶任务已经启动但 Enable 为 false 时，会跳过设置 IsProduct 的生产分支，仍可在未停止时放行。S2 取得的对象若仍为 false，就会在满足输送与固晶启用条件时按 S2 重建。此时 S2 中的实例不再是先前树，运行信息由新构造产生。这个分支与前面的正常双工位路径需要分别描述；它也不等同于机器硬件配置中没有点胶系统。

手动移送还存在目标 TU 为空时补建再刷新的分支；示教中的 JudgeTuExist 会按用户在设备界面中的选择重新建立对象；轨道物料搜索也可能补建或刷新。它们都说明“区段已经有一份 TU”不等于“后续入口一定保留这一份 TU”。需要追踪实例连续性时，应检查实际进入的条件分支。主动清空的三个入口另见 6.10 节。

IsProduct 单独不能作为生产履历凭据。Mapping 显示界面在载入已有 TU 时会把它设为 true，即使此前没有经过 S1 生产入口。分析该标记时必须包含界面入口。其设计上的实际意义是影响后续创建判断，不能单独用来证明某个机械加工过程已经发生。 [9]配方样本 Mapping 与生产 Mapping 的区别见 5.6 节。

## 6.6 物料搜索与人工补料怎样改变 TU

物料搜索用于重新对应实物与区段中的当前 TU；示教和空跑还可从辅助入口建立新对象。表 6-2 先区分这些入口的用途，6.7 节集中说明 S1/S2 搜索和人工确认的完整条件，避免把辅助新建当成历史记录恢复。 [8] [9]

表 6-2 搜索以外辅助入口的对象处理

| 入口与条件 | 对象及坐标变化 | 记录与有效性 |
| --- | --- | --- |
| 待下料段/下料段；人工确认有料且 TU 为空 | 有料状态可保留，但该分支未补建 TU | 有料状态不保证当前 TU 非空 |
| 示教 UpdateTransportUnit | 以“示教”原因新建运行树 | 替换旧引用；构造完成不表示实际加工 |
| DryCycle 来料与放行 | 分别新建 S1/S2 对象并发送相应信号 | 不是实物跨工位沿用同一 TU 的路径 |

S1 搜索、S2 搜索和人工补料的完整分支见 6.7 节。示教会替换运行树，手动交接仅在相应空引用分支补建；对象新建后若又执行 Refresh，其工位偏移需按最后一次几何操作判断。 [8] [9]

## 6.7 搜索与人工确认的完整对象契约

物料搜索的输入有三项：检测是否有料、区段是否已有 TU、操作者是否确认实物存在。人工确认有料不会恢复丢失的运行记录；只有沿用原对象的分支才能保留旧 Info 和工序。表 6-3 先列 S1 和配置输送的 S2 共同分支，再列两侧差异。 [8]

表 6-3 搜索结果与人工选择对当前对象的作用

| 检测与当前引用 | 人工选择 | S1 与配置输送的 S2 结果 |
| --- | --- | --- |
| 检测有料；TU 非空 | 无需确认 | 置 HasMaterial；同一 TU 执行 Refresh；Info 和步骤保留，坐标及相关标记刷新 |
| 检测有料；TU 为空 | 确认有料 | 分别按 S1 或 S2 参数新建；置 HasMaterial；新 Info 及步骤来自当前配方 |
| 检测有料；TU 为空 | 确认无料 | 置 NoMaterial；TU 仍为空 |
| 检测无料；TU 非空 | 确认有料 | 保留原 TU，置 HasMaterial；此分支不执行 Refresh；S1 另按分段位置决定回移 |
| 检测无料；TU 非空 | 确认无料 | TU 清空并置 NoMaterial；旧对象不再是当前区段产品 |
| 检测无料；TU 为空 | 无需确认 | 保持空引用并置 NoMaterial |

两侧在“不反搜＋自动工作＋已有 TU”时均可提前返回，跳过上述检测和状态改写。手动搜索传入 false，不满足这一自动免检条件。S1 正常走到方法末尾时将 IsTuInDispense1 置 true；若检测无料而人工确认有料，且原标记为 false，先调用 System1BackMoveDistance，再置 true。它保留同一对象与记录，不清定位或测高标记。若检测有料而已有 TU，走的是 Refresh 后置 true，不能画成同一回移路径。

S2 未配置输送时改用开真空检测。检测成功且 TU 非空只置 HasMaterial，不刷新坐标；TU 为空时由人工确认决定新建或保持为空。检测失败但已有 TU，人工确认有料则保留；确认无料则清空。检测失败且 TU 也为空，Retry 回到 RetryCommand 重新检测，退出则置无料、清引用并关闭真空。这些分支没有与 S1 相同的分段位置标记。

待下料段和下料段的有料/TU 空确认分支没有补建产品树；上料段主要维护检测状态。因此调试时要同时查看 HasMaterial 和 TU 引用。界面的搜索按钮只是进入上述控制器方法，不增加“恢复历史产品”的语义。

## 6.8 分段点胶从消息划分到坐标回移

Divide 把一轮点胶消息分为两组。控制器以 GetFirstMatrixIndex 形成索引集合：常规产品取矩阵前 floor(列数/2)列，异形产品取基板节点局部 X 小于 0 的节点序号。排序结果先执行集合外消息，再插入 InteractiveAction，最后执行集合内消息；不能依据方法名把第一组直接叫作“前半数量”。非 Divide 分支不执行这次划分。 [3] [8]

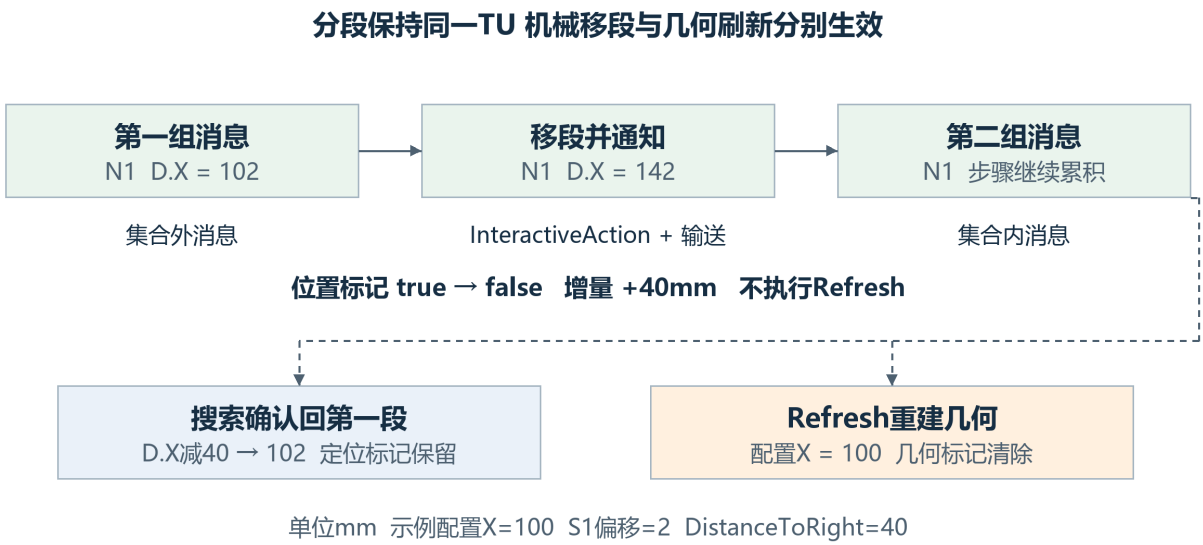



图 6-5 分段过程保持对象并改变位置与消息阶段

从左到右阅读。N1 表示同一 TU；D 是 TU 当前平移，d 是 DistanceToRight。中间移段仅增量改变 D.X，不调用 Refresh。虚线表示搜索或后续跨工位交接可能进入的独立路径。图中两组表示索引集合内外，不表示相等数量。

第一组结束后，InteractiveAction 仅在 Divide 且 IsTuInDispense1 为 true 时移到安全高度、发出 AllowDispense1TransferToDispense2Signal，再等待完成信号。输送控制器要求当前段有料；自动路径等待放行，手动路径还要求 Divide 且位于第一段。机械移动结束后置位置标记 false，给 TU 的 X 平移加 DistanceToRight，并将 IsNeedPreDispense 设 true；任务取得 Success 才发送 TransferToDispense2FinishedSignal。交互动作恢复后置 IsFirstDispense，随后执行第二组消息。

移段过程没有创建或替换 TU，也没有统一清除 IsVisioned、IsMeasureHeight 及焊点步骤。第二组是否定位取决于该对象自己的标记和动作条件；TU 级定位在 Divide 的 DoWork 中直接跳过，不能写成整棵树自动重新定位。IsNeedPreDispense 与 IsFirstDispense 服务预点胶阶段，也不等同于清空定位结果。

表 6-4 分段位置与刷新操作的连续变化

| 节点 | 对象与 X 平移 | 标记及工序 |
| --- | --- | --- |
| 第一段处理 | N1；D.X | IsTuInDispense1=true；已完成步骤留在 N1 |
| 移到第二段 | 仍为 N1；D.X+d | 位置标记 false；预点胶标记更新；定位/测高和步骤不统一清空 |
| 搜索无料而人工确认有料 | 仍为 N1；若原在第二段则减 d | 末尾位置标记 true；Info 及步骤保留 |
| 搜索检测有料且已有 TU | 仍为 N1；按配方重建坐标 | Refresh 清几何标记，位置标记置 true；不在 Refresh 内重新加工位偏移 |
| S1 交给 S2 | S2 接收 N1 后 Refresh | 产品步骤保留；两侧偏移和定位/测高标记清除 |

说明输入取配方 TU 平移 X=100mm，S1 偏移+2mm，DistanceToRight=40mm。S1 构造后为 102，移段后 142；搜索中的回移得到 102。若改走 Refresh，则得到配置值 100 并清工位偏移标记。三种数值对应不同操作，不能用一次减 40 替代刷新。若在移段的机械步骤中断，位置标记和 X 增量可能尚未更新；InteractiveAction 本身没有重建几何或统一回滚的出口。

## 6.9 其他输送入口如何处理对象

表 6-5 输送入口的对象接收与退出方式

| 入口 | 正常出口中的 TU 处理 | 需要保留的差异 |
| --- | --- | --- |
| 自动上料到上料段 | 创建 S2 参数 TU，供后续入口使用 | 后续 S1 可重新构建生产对象 |
| 上料皮带到点胶 | 接收上料段引用后 Refresh，清源 | 不能据此替代 S1 生产 IsProduct 判断 |
| 上料仓到点胶 | 当前点胶 TU 为空时按 S1 新建，随后 Refresh；清上料段 | 不执行相同的源 TU 引用交接 |
| 点胶第一段到第二段 | 保持同一 TU，增量改 X | 不 Refresh、不转移到另一个区段引用 |
| 点胶到固晶 | 目标接收源引用后 Refresh，再清源 | 手动且目标空时有补建分支 |
| 固晶到待下料 | 先接收引用并置目标有料，再?.Refresh，最后清源 | 空 TU 可跳过 Refresh；目标有料状态写入较早 |
| 待下料到下料皮带 | 接收引用后直接 Refresh，再清源 | 异常仅回写等待段有料状态，不恢复整套引用 |
| 待下料到料仓 | 清等待段 TU 并更新区段状态 | 没有把旧树赋给下料段的对应代码 |
| 下料皮带到自动下料 | 完成后清下料段 TU 并置无料 | 产品对象退出当前输送区段 |

表中列的是正常出口的程序数据操作。各入口的检测、手动确认及停止条件仍分别生效，不能因出口最终都叫 Success 而推断机械过程、刷新次序或恢复能力完全相同。 [8]

## 6.10 主动清空载具与局部清空

主动清空用于撤销区段当前持有的产品入口。InitializeTS 清除上料、点胶、固晶、待下料和下料五个区段的 TU 引用，并把这五处 SubSectionState 置为 NoMaterial；两个局部方法只处理点胶段或固晶段。它们会执行相应安全移动和松夹等机械操作，但清引用本身不证明实物已被拿走。 [17]




图 6-6 整线清空与两种局部清空的对象范围

每列是一项独立操作的正常数据出口，不能同时叠加阅读。实心标记表示该区段 TU 设为 null 并置 NoMaterial，空心标记表示本方法没有改写该区段。三个入口均先检查机器状态并要求确认。

表 6-6 清空与刷新在对象和记录上的差异

| 操作 | 当前对象入口 | 几何与记录 |
| --- | --- | --- |
| 整线清空 | 五个区段 TU 均设 null | 区段不再持有旧树；没有逐节点清 Info 或完成字典 |
| 只清点胶 / 只清固晶 | 仅所选区段 TU 设 null | 其他区段引用不由该方法清除 |
| Refresh | 保留当前 TU 和后代对象 | 替换坐标，清定位 / 测高及偏移标记，保留工序记录 |

三个控制器方法在机器 Working 或 Pause 时直接返回；通过后还要取得 Yes 确认。快捷菜单另加 IsStop 检查，手动页和其他调用方仍以实际入口条件为准。拒绝确认时不执行清空分支。正常分支在内存引用和区段状态修改之后调用 Save，没有统一事务回滚；保存异常不能解释为旧 TU 已经恢复。

清空是解除当前区段对产品的持有关系，不是销毁所有别名。界面、动作或调试代码如果此前缓存了旧 TU，引用仍可非空；后续应重新从区段取得当前对象。原方法也未统一重置 SubSectionTransferState、信号池或动作控制器消息，不能把“清空载具”作为这些状态全部复位的证明。

## 6.11 辅助对象与补偿工作模式

相同构造函数还服务于示教、局部测试与补偿工作。判断记录是否连续，先确认新对象被赋给了区段，还是仅由窗口或局部变量持有。辅助流程中的新 TU 不自动继承现有产品 Info，也不能只因构造参数含工位就认定完成了区段交接。 [18]

表 6-7 辅助入口的对象拥有者与替换条件

| 入口 | 拥有者与创建条件 | 记录及偏移 |
| --- | --- | --- |
| JudgeTuExist 在线示教 | 已有区段 TU 时可重建；IsProduct 为 true 先请求确认；为空则提示并返回空 | 按本侧系统参数新建；旧运行记录不复制 |
| JudgeTuExist 离线示教 | 直接返回窗口使用的新 TU，不赋给生产区段 | 字符串构造，不自动加工位偏移 |
| 移动到焊点窗口 | Init 创建窗口自己的 TU | 不能据此推断区段树已被替换；位置辅助方法仍可能读取区段 TU |
| 点胶辅助测试 | 满足前置检查后在局部变量中创建本侧 TU | 新 Info；S1 构造后再调本侧偏移受幂等标记保护 |
| S2 补偿工作模式 | 来料后先用字符串构造替换固晶段 TU | 后续是否再次按 S2 重建，由 Enable 及输送配置决定 |

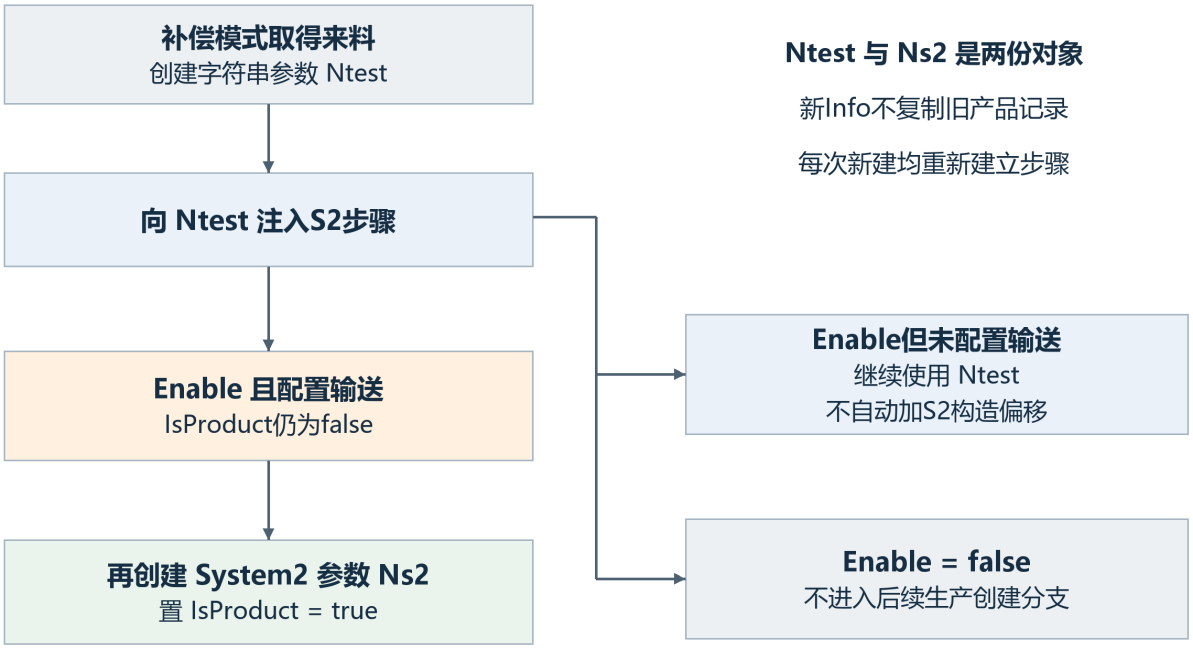



图 6-7 补偿工作模式中的两次创建判断

只展开 IsCompensateWork 为 true 且已取得来料的分支。Ntest 与 Ns2 表示不同的新实例；Enable 为 false 时不进入生产分支。普通模式跳过第一处创建，仍按 6.4 节处理原 TU。

补偿工作模式下，任务先执行 new TransportUnit("测试")并注入 S2 步骤。若 Enable 为 true 且配置输送，由于这份新 Info 的 IsProduct 为 false，随后再次按 System2 构造并置 IsProduct 为 true；前一份 Ntest 不再是区段当前对象。若启用固晶但未配置输送，则保留字符串构造的 Ntest，不能自动补算 S2 构造偏移；Enable 为 false 时也跳过后面的生产创建分支。

实验窗口中的测试创建属于辅助归属，不作为正常产品交接。需要恢复生产时，应核对当前区段 TU、IsProduct、实际几何以及动作消息，不能把窗口中的样本对象或测试记录当作上一份生产产品的延续。

# 7 主要接口与使用约定

## 7.1 创建和刷新操作的选择

调用 TU 接口前，先明确本次操作要改变的是什么。新建用于建立新的运行身份和记录；Refresh 用于结构仍相符时重建几何；从当前工位重新取 TU 用于跟随区段引用变化。这三种操作互不等价。表 7-1 按意图给出它们的前提与效果，便于开发和调试采用相同判断。 [1] [8]

表 7-1 创建和刷新接口的语义

| 操作 | 前提 | 主要效果 |
| --- | --- | --- |
| 有参构造 | 配方、父坐标与工序配置可用 | 新建树、Info 与坐标；系统参数构造另外加工位偏移 |
| Refresh | 既有树与当前配置仍匹配 | 保留对象及步骤；重建坐标；清定位、测高和偏移标记 |
| 重新读取当前工位 TU | 当前区段引用已正确更新 | 取得此刻有效的对象入口；不创建、不刷新 |

调用者如果长期缓存 TU 或坐标，应分别处理两个失效点。生产入口重建或区段引用改变以后，旧 TU 可能已不是当前产品；同一 TU 执行 Refresh 以后，旧 CoordinateSystem 也可能已经脱离产品树。保存长寿命引用的代码，需要明确它服务于历史记录还是当前控制，不能仅凭引用非空判定可用。

## 7.2 产品地址查询

产品地址有两种常用含义。Index 表示对象在产品结构中的序号，WorkIndex 表示加工编排使用的工作序号。普通顺序下二者可能相同；蛇形顺序下，同一个数字会指向不同对象。图 7-1 用输入 4 展示这个差别。 [1] [3]

TU 的 GetSubstrate、GetModule 按 Index 查询；GetBondPosition 先按祖先 Index 查到基板和基岛，再按焊点名称查找。动作控制器另按 WorkIndex 解析当前对象：S1 直接查找，S2 在 InitCurrentMessage 中逐层匹配。因此，传入接口前要先确定数字的含义。表 7-2 列出各接口约定，正常查找以集合已建立为前提。 [1] [3]

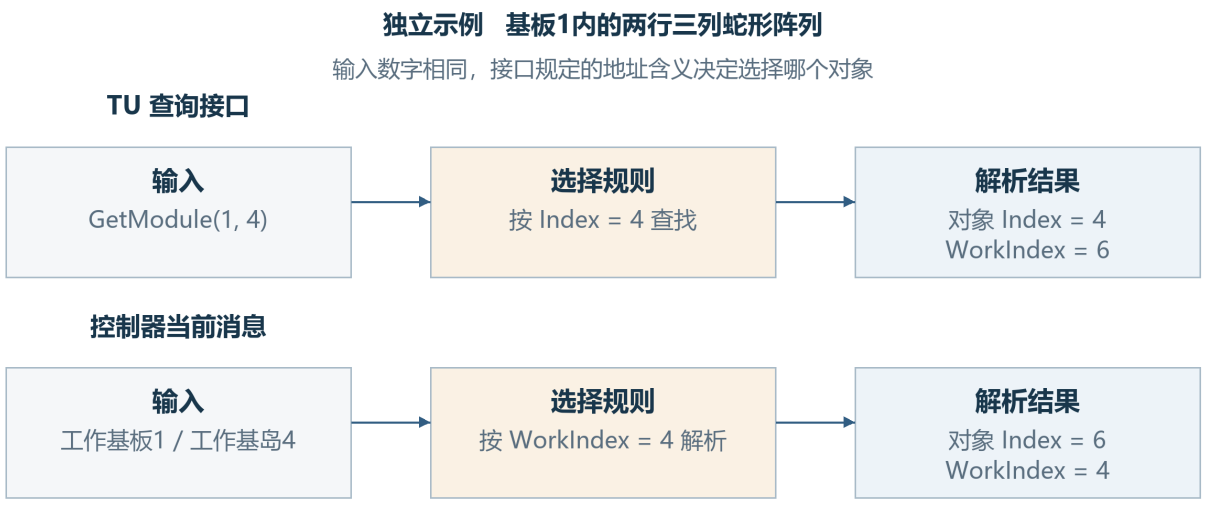



图 7-1 同一输入数字在两类查询路径中的不同含义

沿用图 3-2 的独立蛇形阵列：TU 接口的参数 4 指 Index，控制器消息中的基岛工作序号 4 指 WorkIndex。前者选对象 4，后者选对象 6。两条路径的结果不同，来源是接口契约不同；本图不改变贯穿实例的产品数量。

表 7-2 TU 地址查询接口的输入输出

| 接口 | 输入的地址含义 | 返回与失败行为 |
| --- | --- | --- |
| GetSubstrate(index) | 基板 Index；常规构建从 1 开始 | 返回 Substrate 或 null；集合存在时负数抛 ArgumentOutOfRangeException |
| GetModule(substrateIndex, moduleIndex) | 基板 Index 与该板内基岛 Index | 返回 Module 或 null；先查父基板，父存在后才检查基岛负数 |
| GetBondPosition( substrateIndex, moduleIndex, bondPositionName) | 祖先 Index 与该岛内焊点名称 | 返回 BondPosition 或 null；祖先存在后才检查名称是否为 null |

图 3-2 的蛇形例能显示这一差别。假设该阵列位于基板 1 中，GetModule(1,4)取得 Index 为 4、WorkIndex 为 6 的基岛；动作消息中的基岛工作序号 4 则解析到 Index 为 6 的对象。同一个整数在两条调用路径中有不同含义，普通顺序下两者相同不代表接口可以互换。

查找失败时，返回 null 或抛异常由实际分支决定。基板集合为 null 时直接返回 null；集合存在后才检查负序号。基岛查询找不到父基板时先返回 null，不再检查下层序号。 [1]

焊点查询先确认祖先存在，再检查焊点名是否为 null；空字符串会进入查找，并以未找到结束。调用者需分别处理未找到与参数错误。 [1]

GetAllBaseMatter 取得根以下的实际对象，包含基板、基岛和焊点而排除 TU 根。GetAllBondPositions 则按名称收集焊点，但两层循环只到常规 Count 减 1：本例查找 A 点应有 6

个，该方法却只遍历基板 1 的前两个岛，得到 2 个。其数量接口读取常规全局 Count，也不能替代异形树实际计数。附录 C 集中记录这些现状。

S2 当前对象解析还使用了缓存。InitCurrentMessage 在匹配成功时写入 CurrentSubstrate、CurrentModule 和 BondPosition，但没有先统一清空这些字段；本轮地址未命中时，非空结果可能仍来自上一次消息。因此“当前对象非空”不能单独证明它与本轮消息一致，完整配置与地址匹配仍是调用前提。 [3]

## 7.3 步骤记录与动作记录

InjectSteps 按当前已启用工序补入缺失键，已有键和值都保留。步骤完成写入方法也能在键不存在时创建该键；相反，是否需要执行的单步查询直接读取字典，缺键会抛出异常。因此先注入再查询是当前生产链的重要前提，不能因为写入能够补键，就推断所有读取都能容忍缺键。 [3] [4]

FinishCurrentActionNode 只改变当前工作消息的 Finished 状态。焊点的 S1 或 S2 完成写入由相应动作在成功路径调用。控制器 Finish 用于恢复消息编排供后续产品使用，也不会自动恢复焊点未完成状态。要实现重做、跳过或人工确认，应明确修改哪份记录以及该对象是否仍是当前产品，不能借调用 Finish 含混处理。

## 7.4 坐标与状态接口

SelfPosToG0 接受本层坐标中的点，G0PosToSelf 接受 G0 中的点。两者应成对采用同一条当前父链。对焊点自己的原点求全局位置时输入(0,0,0)，不是把已经含有局部平移的配置点再输入一次；后者会重复叠加本层位置。更新上层坐标后，下层转换自然使用新位姿，无需逐个重写后代局部坐标。 [5]

工位偏移方法以各自标记防止本侧重复叠加；清标记不等于已经恢复几何，恢复几何由 Refresh 等操作完成。分段前移和回移是增量加减，需要成对理解其调用条件。定位与测高标记的清除同样只表示下一阶段需要重新处理，不会删除 Info 中的历史结果数值。

SetMatterDisable、SetMatterOnlyDispense、SetMatterOnlyBond 与 SetMatterEnable 向下层传播工艺参与范围。基板与基岛的聚合属性随后反映下层状态；TU 本身没有同样的聚合覆盖。修改工艺参与范围不会自动等同于清除工序结果，界面是否同时清步骤还取决于选择的是焊点还是上层对象。 [1] [9]

## 7.5 步骤与转换接口的调用契约

下表把常用输入、返回值和前提集中用于查阅。返回 void 只说明方法不返回结果对象，并不表示没有副作用。控制器复位针对工作消息，焊点步骤写入针对产品记录；调用者应在正确的拥有者上操作。 [3] [4] [5]

表 7-3 步骤及坐标接口的输入输出

| 方法与输入 | 输出与副作用 | 调用约束 |
| --- | --- | --- |
| InjectSteps(transportUnit) | void；在匹配焊点上补缺失工序键并设 false | TU 及下层集合、工序配置应可遍历；已有键保留 |
| IsS1NeedStep(stepName) / IsS2NeedStep(stepName) | bool；返回对应完成值的反值 | stepName 非 null 且本侧字典已有键；缺键抛 KeyNotFoundException |
| S1FinishedStep(stepName) / S2FinishedStep(stepName) | void；本侧字典值置 true；缺失键被建立 | stepName 非 null；成功路径写入；不标记控制器消息 |
| FinishCurrentActionNode() | void；当前工作消息 Finished 置 true | 必须存在当前消息；不代替焊点工序写回 |
| Finish() | void；将本控制器工作消息 Finished 复位为 false | 两侧集合为 null 均返回；S2 集合非 null 时还要求至少一项 |
| SelfPosToG0(point3D) / G0PosToSelf(point3D) | 返回转换后的 AKRSPoint3D；具体转换由机器坐标接口完成 | 输入参考系及方向对应；当前对象和父链可用 |

## 7.6 异形节点反查与位置辅助接口

GetBaseMatterByOppositeSex 接受一份配置节点对象，先比较 TU 根，再遍历基板、基岛和焊点，返回关联到该节点的运行对象。当前判断使用 OppositeSex 对象引用相等，既不按 Name，也不按 Id 或 ConfigName 匹配。找不到时抛异常，不返回 null；它与 7.2 节的 Index 查询是另一份接口契约。 [15]




图 7-2 按配置节点反查运行对象的匹配边界

左侧 N 表示创建 TU 时关联的配置节点对象；右侧 N′即使显示名、ID 和 ConfigName 都相同，只要不是同一引用，仍不能命中 N 关联的运行对象。图示强调引用身份，不表示两个节点物理位置不同。

异形示教先从当前配置树取得父节点，再调用该接口找父运行对象和目标运行对象。配置树若已替换而 TU 仍沿用旧关联，名称相同也不保证匹配。调用者应确保节点来自这份 TU 所关联的配置树；这是节点反查的前提，不可用配置文件反序列化后的同名对象代替。

表 7-4 位置辅助接口的来源和返回参考系

| System2TUService 方法 | 位置来源 | 实际返回 |
| --- | --- | --- |
| GetSubstratePos | 基板当前 G0 点；相机偏移 XY 减、Z 加 | 继续调用 ConvertG0ToMachinePos，返回运动轴坐标 |
| GetModulePos | 基岛当前 G0 点；相机偏移 XY 减、Z 加 | 返回相机目标的 G0 点，方法内未转运动轴 |
| GetBondPositionVisionPos | 区段 TU 内基岛坐标与焊点配置点 | 先转运动轴再减 HeadToCameraOffset |
| GetBondPositionTeachPos | 区段 TU 内基岛坐标与焊点配置点 | 返回运动轴坐标 |

这些辅助方法均从固晶区段读取 TU。其返回值不能仅凭方法注释中的 G0 或 Bond 字样判断；GetModulePos 与 GetSubstratePos 确实停在不同转换阶段。焊点示教辅助用的是基岛当前父链和 SingleBondPositionConfig 中的配置点，没有直接取该焊点当前坐标原点。因此它不能替代生产动作取得当前焊点位姿的路径。 [19]

这些方法会对部分下层对象缺失返回 null，但不在入口统一保护 TU 为空。调用者需要区分产品入口不存在、地址未命中以及已有位置的参考系。窗口自行创建 TU 也不改变该服务读取固晶区段 TU 的事实，调试时应同时查看窗口对象和服务实际拥有者。

# 8 一致性约束及失败恢复边界

## 8.1 配方与对象树及步骤的一致性

TU 运行依赖当前对象树、配置与工序定义共同成立。只改坐标而保持结构时，可以通过刷新重新建立几何；改变层级数量、焊点名称或异形关联以后，旧树可能不再满足配置索引条件。Refresh 沿既有集合执行，不按新数量重建整棵树，也不重新应用工艺状态或重新注入步骤。因此它不能承担任意配方迁移。 [1] [2]

工序注入也没有完整的配置迁移语义。重命名工序后，新的键可以被补入，旧键仍留在字典；完成属性仍遍历字典已有项。改变焊点名又会影响匹配与当前对象解析。研发评审应同时检查定义、消息和运行字典；调试时看到旧工序键，应先判断是否来自旧对象及旧配置阶段。

## 8.2 交接成功与交接中断

正常自动交接完成移送后更新产品引用和区段状态；报警选择也会影响这条路径。两处卡料报警的 Ignore 只跳出选项分支，仍能继续夹紧、引用交接、Refresh、状态保存并返回 Success。因此 Success 表示当前程序路径完成，不能单独证明所有物料检测都正常通过。 [8]

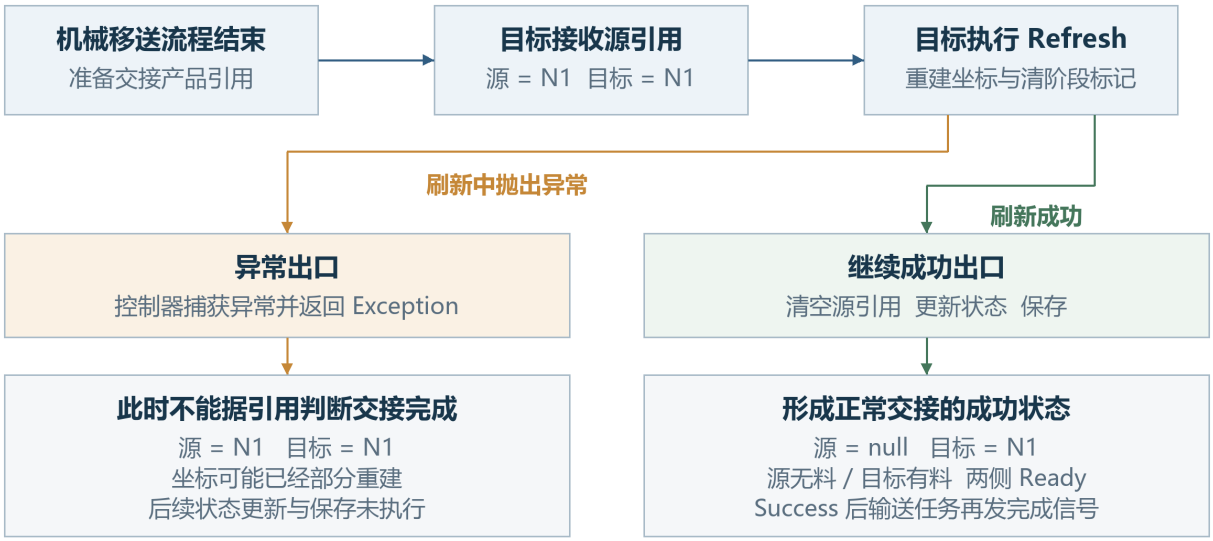



图 8-1 在 Refresh 处中断时的引用和状态边界

本图只考察目标接收源引用后、Refresh 抛异常这一位置。目标和源可暂时同时指向 N1；外层捕获异常并返回 Exception，后续清源、更新成功状态及保存未执行。成功分支按顺序完成这些操作后，输送任务才发送来料完成信号。

这里的关键是“哪个修改已经发生”。目标引用赋值发生在 Refresh 之前；坐标又按层逐步重建，因此异常出口既不能视作完全未交接，也不能视作正常完成。当前方法没有自动把引用、坐标与区段状态恢复成同一时刻的快照。 [8]

后续处理需要重新满足接口前提：实物位置与检测状态要和区段状态对应，当前 TU 及几何要属于本次产品，控制器返回与来料信号要对应本次交接。图 8-1 解释现有实现能证明哪些状态；源码没有形成的统一恢复协议，在本册中保留为设计边界。

重新启机时，输送启动根据配置刷新已有产品并搜索物料，界面也允许在物料不一致时按条件补建或清除 TU。软件引用、区段有料状态、传感器检测与实体位置是几个需要重新对应的事实。这些入口分别处理检测与软件状态，不提供完整产品运行履历的自动恢复协议。

## 8.3 人工操作改变哪些记录

生产 Mapping 界面的停用、正常和成功按钮会检查机器是否处于工作状态。在允许修改时，对焊点选择和上层选择的处理不同：选择焊点后设为正常，会同时把两侧步骤恢复为未完

成；设为成功，会把两侧步骤设为已完成；停用该焊点也会重置两侧步骤。选择基板或基岛主要向下传播参与状态，该按钮分支并不同时按焊点分支重置所有后代字典。 [9]

因此，对一个基板设为正常，不能直接推断其中每个焊点已进入可重做状态；对一个点人工置成功，也不能据此认定机器实际执行了点胶与固晶。设计中的人工状态调整与实物加工证据分开保存和解释，才能避免界面显示与调试判断相互混淆。

## 8.4 保存的范围决定可恢复的范围

交接后的“保存配置”不等于保存了可恢复生产的完整快照。区段中的当前 TU 和坐标使用 JsonIgnore 等排除机制，保存文件不同时包含完整产品树、父坐标链接、工序字典和控制器消息。 [8] [10]

独立 TU 存储接口也没有在本文追踪的正常生产主链中形成完整的保存、加载及重新绑定过程。因此，恢复能力要按实际保存内容判断，不能仅凭保存调用已经执行作出结论。 [10]

Mapping 导出用于产品结果表达：可写入配方、批次、产品号和面别等信息，并按基岛的失败与 S2 完成情况输出相应标记。它不序列化全部 TU 运行信息，也不恢复机器父坐标关系。要据文件恢复生产，必须另外定义重绑定与进度恢复协议；不能从“已有导出文件”推断当前实现已经具有断电续作能力。 [2] [7]

这些边界属于现有设计可承担的范围。它们不否定正常成功路径的对象连续性，但要求调用者在重建、失败与恢复场景重新核对前提。附录 C 集中记录接口行为与设计直觉不一致的地方，供后续维护时定位。

## 8.5 Mapping 导入重建的是哪一份产品

Mapping 载入先按当前配方创建新的 TU，再把文件内容映射到新树。它没有反序列化并复原先前全部运行对象。配方名称、批次、产品号和面别从文件写入；树的数量、坐标和工序定义仍来自载入时的配置。 [14]

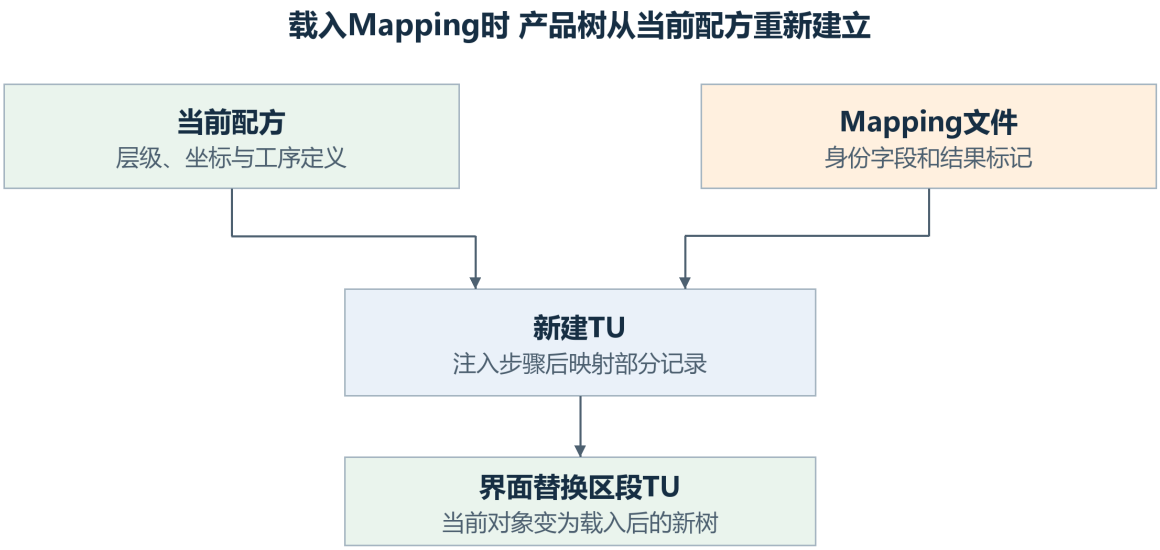



图 8-2 Mapping 载入从当前配方生成新树并恢复部分记录

文件提供身份及结果标记，当前配方提供结构、几何和工序。界面将载入结果赋给目标区段；旧树的对象身份和实时坐标没有被直接带入。

表 8-1 Mapping 载入的结果映射

| 文件内容或操作 | 标准 Mapping 路径 | 芯片代码 Mapping 路径 |
| --- | --- | --- |
| X 标记 | 对应基板或基岛设为停用 | 对应基板或基岛设为停用 |
| 完成标记 | 2 使对应焊点两侧工序均置完成 | 非 X 且非 1 的值使两侧完成，并写 ComponentCode |
| 1 或未完成位置 | 保留新建树的初始步骤状态 | 1 保留新建树的初始步骤状态 |
| 界面载入成功 | 把新 TU 赋给固晶段或所选区段 | 使用相同的界面替换入口 |

以上结果匹配还依赖当前行列、配方名称及文件布局相容。文件缺失返回 null；内容行数、索引或对象查找不满足条件时可能抛异常，不能按一次读取调用推断全部文件都可恢复。显示界面还可能把 IsProduct 设为 true，这会影响后续生产入口是否重建；它不证明产品实际完成过哪道加工。 [9] [14]

## 8.6 交接在不同位置中断后的可见状态

以点胶到固晶为基准，将退出位置与已经发生的修改放在一起。返回值说明控制流出口，区段状态和 TU 字段说明已经发生的数据变化，两者要同时读取。表 8-2 中的“尚未”仅针对本次调用，不覆盖之前调用残留的状态。 [8]

表 8-2 交接的停止 重试与部分提交边界

| 退出或选择位置 | 已发生的数据变化 | 返回与再次进入 |
| --- | --- | --- |
| 入口有料条件不满足 | 本次尚未交接引用或改坐标 | NoStart；输送任务后续循环再判断 |
| 自动等待放行被停止 | 本次未进入 retry 段 | Abort；任务请求停机 |
| 源侧卡料选择 Retry | 源移送状态已为 Alarm；TU 仍在源侧 | 跳回 retry 重做机械段；没有回滚已执行物理运动 |
| 目标侧卡料选择 Retry | 源和目标可能已进入移送/报警状态；引用尚未交接 | 同样跳回 retry，不恢复 TU 快照 |
| 任一卡料选择 Abort | 本次未到引用交接；保留已写移送状态 | Abort；不会执行后面的清源和成功保存 |
| 任一卡料选择 Ignore | 报警已发生；继续夹紧及后续数据处理 | 可继续到 Success；不是清除或补做所有检测 |
| 目标接收后 Refresh 异常 | 源与目标可同指 N1，坐标可部分重建 | Exception；尚未清源，无统一撤销 |
| 清源与状态更新后 Save 异常 | 目标已持有 N1，源已为空，内存状态可能已完成交接 | Exception；不能按返回失败重放整套数据操作 |

其他入口需保留自己的次序：固晶到待下料先置目标 HasMaterial，再刷新；待下料到皮带的 catch 只恢复等待段的 SubSectionState，不恢复目标引用、旧坐标或所有移送状态；待下料到料仓也只有这项局部状态恢复。当前代码没有可套用于全部输送入口的事务回滚。恢复前应重新对应实物区段和当前对象，不能只凭一个 Abort 或 Exception 推断产品仍在调用前位置。

## 8.7 角度示教的写回次序与取消边界

两点角度窗口接收一个运行坐标对象，两次位置都表达在该对象的上级参考系中。在线通过检查后，窗口先把 RealDegree 写到传入对象的 Degree，再返回 OK 并调用配方 Save。调用方随后还可能把角度复制到另一层运行对象或配置中。因此窗口修改对象、调用方修改目标以及配置实际保存时刻需要分别阅读。 [20]

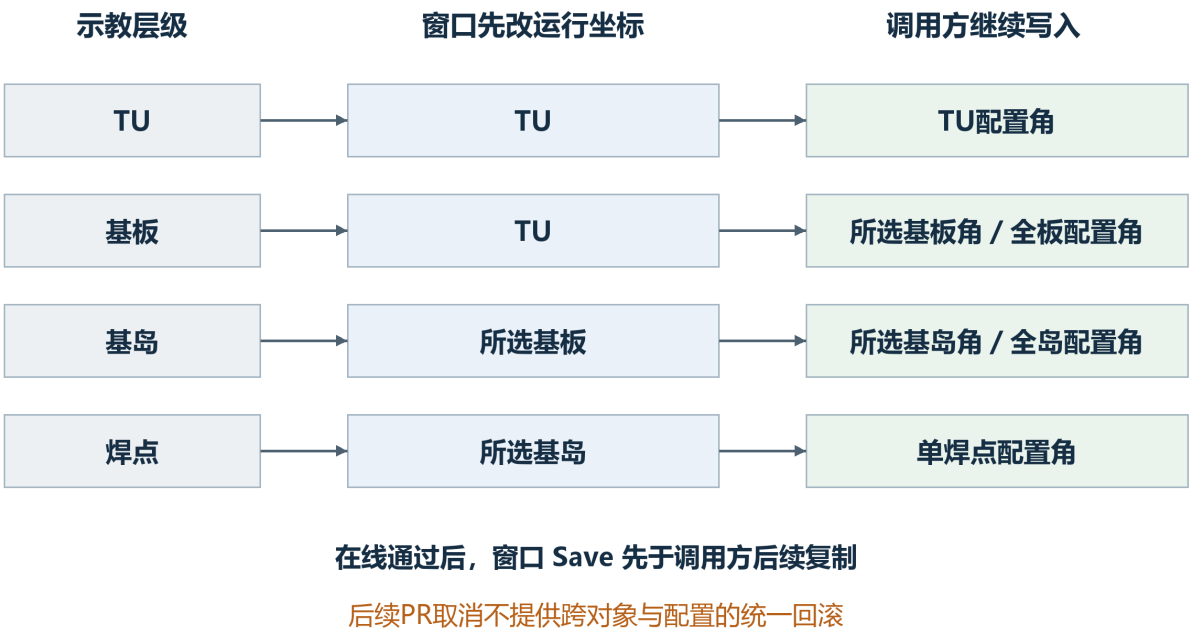



图 8-3 两点示教窗口与调用方的不同写入对象

四行分别对应 TU、基板、基岛和焊点示教。实线表示在线通过后窗口先修改传入的运行坐标，后续箭头表示调用方在相应成功条件下继续复制。窗口中的 Save 发生在调用方后续赋值之前，不能把这些写入视为一次原子提交。

表 8-3 各层角度示教的传入对象与后续目标

| 示教入口 | 传入窗口的坐标 | 调用方后续写入 |
| --- | --- | --- |
| TU 整片或 Divide | TU 运行坐标 | TU 配置角；Divide 在左右 PR 示教之前已执行这一步 |
| 基板且 AssistantAllRotation | TU 运行坐标 | 所选基板运行角；PR 成功后复制到全部常规基板配置角 |
| 基岛且 AssistantAllRotation | 所选基板运行坐标 | 所选基岛运行角；PR 成功后复制到全部常规基岛配置角 |
| 焊点且 AssistanceAngle | 所选基岛运行坐标 | 单焊点配置角；进入角度窗口前已先将该配置角置 0 |

在线窗口要求两点 X 跨度至少 1mm；角度按 atan(ΔY/ΔX)计算，绝对值不超过 5°才接受，Degree 按 rad 保存。取 P1=(0,0)mm、P2=(20,1)mm 时，角约 0.0499584rad，即 2.862405°。窗口成功先改传入对象，因此基板示教期间 TU 运行角也可能已改变，不能把表 8-3 的第三列当作唯一写入。

X 跨度不足或角度过大时，窗口回到前面的步骤且不执行本次成功写入。离线分支直接返回 OK，不计算新角，也不在该分支写 Degree；调用方仍可能继续复制 RealDegree，不能把离线 OK 当作取得有效角度。后续 PR 窗口取消或返回非 OK，也没有覆盖前面运行坐标及配置写入的统一回滚。

因此，示教完成后的使用契约是重新确认目标层级的运行角、配置角及保存结果。Refresh 会把运行几何重新取自配置；它能消除未写入配置的临时运行角，却不能撤销已经写入或保存的配置角。具体两点采集与重试算法见纠偏过程分册，本节保留 TU 对象及配置受到的影响。

# 9 贯穿实例与设计核对

## 9.1 固定一组完整输入

本章把前文的结构、坐标和生产链合为一个实例。采用常规模式，基板和基岛均显式启用多件；工艺参与范围全部 Enable，全部 EntityState 初始为 Process。图 1-1 展示的 12 个焊点与下表输入对应。

表 9-1 贯穿实例的结构与坐标输入

| 层级 | 数量或名称 | 父层中的位置与角度 |
| --- | --- | --- |
| Transport | 机器中一个命名父坐标 | 本例到 G0 为零平移、零角度 |
| TU | 1 | (100,200,0)；角度 π/2 |
| 基板 | 2；一行两列 | 1：(20,0,0)；2：(80,0,0)；角度 0 |
| 每板基岛 | 3；一行三列 | 1：(0,5,0)；2：(10,5,0)；3：(20,5,0)；角度 0 |
| 每岛焊点 | A、B | A：(10,5,0)；B：(14,5,0)；角度 0 |

两层工作方向选择普通从左到右顺序，Index 与 WorkIndex 一致；本例不启用分段移动。S1 工位偏移取(2,−1,0)，S2 偏移取零。启用 D_A、D_B、T_A、T_B 四个工序，分别匹配 A、B 两点及 S1、S2 系统。每个 A 点的两侧字典初始为 D_A=false、T_A=false，B 点相应使用 D_B 与 T_B。图 3-2 的两行三列蛇形阵列仅用于解释序号，不参与本例计算。

## 9.2 从 S1 创建到 S2 处理的连续快照

自动来料后，S1 入口发现旧对象 IsProduct 为 false，于是创建 N1 并设为 true。选取基板 1 / 基岛 2 / A 点。应用 S1 偏移后，其局部原点在 G0 中为(92,239,0)。再假设基板 1 测高差为 +0.12mm，且除此之外没有定位修正，该点变为(92,239,0.12)。完成 D_A 后，A 点的 S1 记录为 true。

表 9-2 同一 A 点在几个阶段的状态快照

| 阶段 | 运行实例与标记 | 位置 / 记录 |
| --- | --- | --- |
| S1 入口后 | N1；IsProduct=true | G0=(92,239,0)；D_A=false；T_A=false |
| S1 测高与点胶后 | 仍为 N1 | G0=(92,239,0.12)；D_A=true；T_A=false |
| 交接 Refresh 后 | 仍为 N1；IsProduct 仍为 true | G0=(90,240,0)；D_A=true；T_A=false |
| S2 成功处理 A 点后 | 仍为 N1 | 本例无额外几何修正；D_A=true；T_A=true |

第三行是本例最容易误读的一步。工位偏移标记被清除，几何恢复为配置链，所以 G0 回到(90,240,0)；基板 Info 仍可读到旧的 0.12 测高差，但 IsMeasureHeight 为 false，当前坐标不再包含那次增量。S2 入口看到 IsProduct 已经为 true，保留 N1。不能在这里再自动加一次 S2 构造偏移，因为这个正常分支没有执行 S2 新建。

本例把 S2 偏移设为零，并假定 S2 后续没有额外定位、测高或局部补偿，目的是单独观察 TU 交接机制。实际设备若启用这些流程，必须在其发生层和发生时刻继续记录坐标变化。最终贴装位置还涉及动作层的工具、芯片等补偿，不能把本例焊点原点直接当成所有生产条件下的最终贴装命令。

当 T_A 完成时，只能说明这一 A 点的这一项 S2 工序已经完成；其他 11 个焊点仍按各自字典判断。全部动作结束后，S2 控制器复位工作消息，当前 TU 的 D_A、T_A 记录仍在。产品成功移向后续区段时保持引用交接，离开最终区段后源引用被清空。

## 9.3 异形产品的连续对象与坐标例子

异形说明输入取根下两块基板，基板 1 在(20,0)mm 且有 2 个基岛，基板 2 在(80,0)mm 且有 3 个基岛；各岛只有 A 点。根到 G0 平移(100,200)mm、角度 0，各基板和基岛角度均为 0。基板 1 的岛 2 在(10,5)mm，A 点在(2,1)mm，因此该 A 点 G0=(132,206)mm。两块基板的各 A 节点均可通过 ConfigName 关联同一工艺配置，各自 Info 与步骤仍独立。

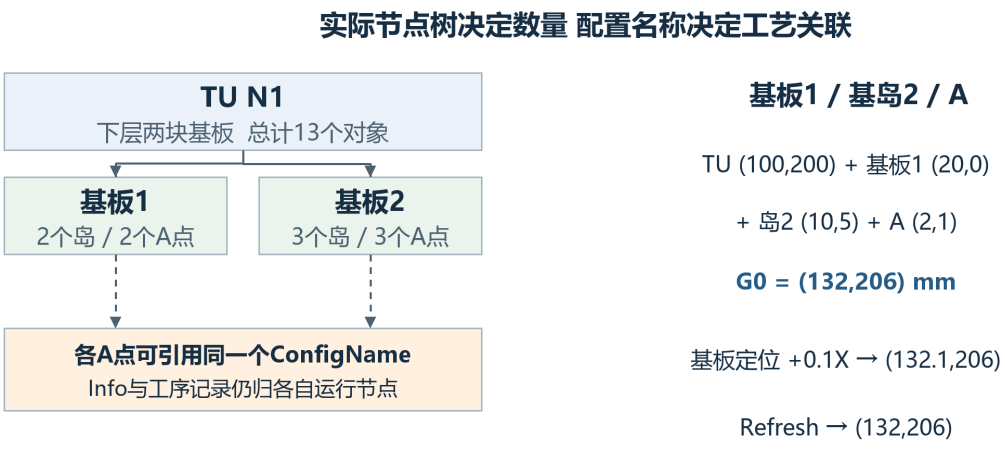



图 9-1 异形节点数量 地址与配置关联分别确定

左侧按实际节点数形成不等长树；右侧跟踪基板 1 基岛 2 A 点。蓝线表示父子坐标，虚线表示配置名称关联。图中长度为说明输入，未按物理比例绘制。

按本例实际树统计共有 1+2+5+5=13 个对象、5 个焊点。查询地址使用基板 Index、基岛 Index 和焊点名称；异形创建的 WorkIndex 初始等于节点序号。若基板 1 定位使其平移从 20 变成 20.1mm，A 点随当前父链变为(132.1,206)mm；Refresh 仍沿既有节点重建坐标，使本例 A 点回到(132,206)mm，原 A 点对象和工序记录保留。若改变节点数量或 ConfigName，须重新核对结构与配置关联，不能把 Refresh 当成重新展开任意新节点树。

## 9.4 用条件变化检查设计理解

如果去掉 S1 启用条件，应重新判断 IsProduct 是否曾置为 true，再判断 S2 是否重建；如果改成异形产品，应从节点树重新统计对象，并检查配置名称关联；如果刷新前后保存了坐标引用，应比较引用是否仍属于当前对象。每个变化都对应前文一个明确的设计边界，而不是只看一次总完成属性。关键方法入口见附录 A。

表 9-3 可用于评审和调试的核对问题

| 问题 | 本设计给出的判断依据 |
| --- | --- |
| 同名 A 点会共用完成记录吗 | 不会；Info 与字典归各运行对象，见 3.1 节 |
| 当前焊点全局坐标从何而来 | 本层到机器父链逐级转换，见第 4 章 |
| 跨 S1 和 S2 是否一定新建 TU | 取决于 IsProduct 及生产入口条件，见 6.4、6.5 节 |
| 刷新是否等于重做产品 | 不等于；几何重建与进度保留分别发生，见 7.1 节 |
| Finish 是否把已加工焊点清零 | 不会；控制器消息与焊点步骤分开，见 7.3 节 |
| 保存或 Mapping 是否足以续作 | 不足以据此作保证；保存范围见 8.4 节 |

# 附录 A 源码依据与阅读定位

### [1] 运行对象与状态

TransportUnit.cs Init / Refresh；BaseMatter.cs GetConfigByType / SetMatterDisable

### [2] 配置与展开

ProductConfiguration.cs InitProductConfiguration / ClearProductConfigurationInstance；TuService.cs CreateMatterCoordinate / UpdateEntity

### [3] 动作控制与步骤

S1ActionNodeController.cs Init / InjectSteps；S2ActionNodeController.cs Init / InjectSteps

### [4] 运行信息

BaseInformation.cs 字段定义；TransportUnitInfo.cs 字段定义

### [5] 坐标调用接口

TuService.cs CreateMatterCoordinate / UpdateEntity；BondModuleController.cs ConvertG0ToMachinePos

### [6] 点胶生产入口

DispenseWorkTask.cs DoWork；S1DispenseActionNode.cs DoWork

### [7] 固晶生产入口

BondTask.cs DoWork；BondActionNode.cs DoWork

### [8] 输送与搜索

TransportController.cs TransferDispenseToBond；BondSubSectionController.cs MapBelt

### [9] 示教与界面入口

TUAssistantHelper.cs JudgeTuExist；UcMainSystem.ShortcutKeys.cs UcMainSystem_KeyDown

### [10] 存储边界

TransportUnitStorage.cs Save / Read；BaseSubSectionProgram.cs UpdateTransportUnit / RefreshTransportUnit

### [11] 定位与测高写回

System1Domain.cs System1MatterVision；System2Domain.cs System2MatterVision

### [12] 启动调度

Machine.Operate.cs CheckOperatingEnvironment / ResetMachineSignal；TransportDomain.cs StartTask

### [13] 身份记录

S1SubstrateIdentityActionNode.cs DoWork；S2SubstrateIdentityActionNode.cs DoWork

### [14] Mapping 导入

TuService.cs ReadTransportMapping / ReadTransportMappingByComponent；FrmTransportUnitMapping.cs FrmTransportUnitMapping / FrmTransportUnitMapping_Load

### [15] 异形配置编辑与节点匹配

FrmTransportUnitEdit.cs BtAdd_Click / BtDelete_Click；OppositeSexConfiguration.cs InitOppositeSexConfigIndex / RemoveOppositeSexConfig

### [16] 配方与生产 Mapping

FrmTuMappingNew.cs FrmTuMappingNew_FormClosed / BtClearAll_Click；FrmTuMapping.cs FrmTuMapping_FormClosed / TsmEnable_Click

### [17] 主动清空载具

TransportController.cs InitializeTS / InitializeTSInDispense；UcShortcutMenu.cs BtInitializeTS_Click / BtClearDispense_Click

### [18] 辅助对象与补偿模式

BondTask.cs DoWork；TUAssistantHelper.cs JudgeTuExist

### [19] 位置辅助接口

System2TUService.cs GetSubstratePos / GetModulePos；FrmMoveToBondingPosition.cs Init / Save

### [20] 角度示教写回

UcEditProductProgramming.Assistance.cs AssistanceTuAdjust / AssistanceSubAdjust；FrmAdjustDegree.cs InitControl

# 附录 B 术语与图表索引

## B.1 常用术语

表 B-1 本文常用术语

| 术语 | 本文中的含义 |
| --- | --- |
| TU / Transport Unit | 一份流转产品的运行对象树根节点 |
| 基板 / Substrate | TU 的直接下层；包含基岛 |
| 基岛 / Module | 基板下的产品子区域；包含焊点 |
| 焊点 / BondPosition | 具体加工位置的运行对象；不表示它必然经过焊接 |
| S1 / S2 | 本工程中分别对应点胶系统与固晶系统 |
| G0 | 坐标转换使用的全局参考；不表示任何数值均为零 |
| Index / WorkIndex | 对象序号 / 加工编排所用的工作序号 |
| Info | 某个运行对象自己的过程信息与结果记录 |
| IsProduct | 参与生产入口重建判断的标记，不能证明已加工 |
| 工作消息 / WorkMessages | 控制器保存的动作编排及其完成标记 |
| 工序步骤字典 | 焊点保存的“工序名→是否完成”记录 |
| Refresh | 沿既有树重建坐标并清除相应流程标记 |
| 区段有料状态 | SubSectionState；HasMaterial 和 NoMaterial 供移送进入条件判断 |
| 区段移送状态 | SubSectionTransferState；表示 Ready、Transfering 或 Alarm 等阶段 |
| 身份识别记录 | IsIdentity 与 IdentityCode 分别保存处理标记和识别代码 |
| 异形节点 | OppositeSex；用节点层级、位置及配置名称描述产品分支 |
| mm / rad | 本文位置计算的长度单位毫米 / 平面角度单位弧度 |

## B.2 图目录

图 1-1 产品层级与一个具体焊点的地址 ..... 4
图 2-1 产品配置 运行模型与动作执行的分工 ..... 6
图 3-1 重复焊点共享配置定义而分别保存运行记录 ..... 8
图 3-2 一组两行三列阵列中的对象序号和工作序号 ..... 9
图 3-3 配置读取与运行数据的持有关系 ..... 9
图 4-1 焊点局部原点逐层转换的中间结果 ..... 13
图 4-2 TU 旋转后的局部轴与焊点全局位置 ..... 14
图 5-1 从产品配置到运行对象树的建立顺序 ..... 15
图 5-2 常规模板与异形节点的展开差异 ..... 17
图 5-3 异形配置编辑与运行对象的两个更新阶段 ..... 18
图 5-4 配方 Mapping 与生产 Mapping 的数据去向 ..... 19
图 6-1 正常 S1 到 S2 路径中对象、几何与进度的有效期 ..... 20
图 6-2 自动点胶到固晶交接的主要调用与信号 ..... 21
图 6-3 工作消息与产品步骤是两套记录 ..... 23
图 6-4 S1 与 S2 生产入口对保留或新建 TU 的判断 ..... 23
图 6-5 分段过程保持对象并改变位置与消息阶段 ..... 26
图 6-6 整线清空与两种局部清空的对象范围 ..... 28
图 6-7 补偿工作模式中的两次创建判断 ..... 29
图 7-1 同一输入数字在两类查询路径中的不同含义 ..... 31
图 7-2 按配置节点反查运行对象的匹配边界 ..... 33
图 8-1 在 Refresh 处中断时的引用和状态边界 ..... 35
图 8-2 Mapping 载入从当前配方生成新树并恢复部分记录 ..... 37
图 8-3 两点示教窗口与调用方的不同写入对象 ..... 39
图 9-1 异形节点数量 地址与配置关联分别确定 ..... 42

## B.3 表目录

表 3-1 主要对象的职责与直接数据 ..... 7
表 3-2 参与本侧工艺时的完成函数规则 ..... 10
表 3-3 配置与位置数据的读写规则 ..... 11
表 3-4 入口标记与过程记录的读写规则 ..... 11
表 5-1 三类 Mapping 入口的数据与保存对象 ..... 19
表 6-1 点胶到固晶移送中的状态与引用 ..... 22
表 6-2 搜索以外辅助入口的对象处理 ..... 24
表 6-3 搜索结果与人工选择对当前对象的作用 ..... 25
表 6-4 分段位置与刷新操作的连续变化 ..... 26
表 6-5 输送入口的对象接收与退出方式 ..... 27
表 6-6 清空与刷新在对象和记录上的差异 ..... 28
表 6-7 辅助入口的对象拥有者与替换条件 ..... 29
表 7-1 创建和刷新接口的语义 ..... 30
表 7-2 TU 地址查询接口的输入输出 ..... 31
表 7-3 步骤及坐标接口的输入输出 ..... 33
表 7-4 位置辅助接口的来源和返回参考系 ..... 34
表 8-1 Mapping 载入的结果映射 ..... 37
表 8-2 交接的停止 重试与部分提交边界 ..... 38
表 8-3 各层角度示教的传入对象与后续目标 ..... 39
表 9-1 贯穿实例的结构与坐标输入 ..... 40
表 9-2 同一 A 点在几个阶段的状态快照 ..... 41
表 9-3 可用于评审和调试的核对问题 ..... 42
表 B-1 本文常用术语 ..... 45
表 C-1 需要保留的实现边界 ..... 47

# 附录 C 当前实现的特定边界

本附录集中列出会影响调用契约和状态判断的特定行为，便于研发评审和设备调试查阅。

表 C-1 需要保留的实现边界

| 位置 | 当前行为 | 阅读影响 |
| --- | --- | --- |
| 按名称遍历焊点 | 两层循环只到常规 Count 减 1 | 末块基板及每板末个基岛漏查；见 7.2 节 |
| 焊点的 S1 完成函数 | S1 非空时还检查 S2 是否为空 | 两侧完成规则不完全对称；见表 3-2 |
| Refresh 及 S2 大数量排序 | 刷新沿已有列表赋序号；S2 可原地排序 | 列表位置不能视为永久地址；见 3.3 节 |
| TU 工艺参与属性 | 根未覆盖基板 / 基岛的聚合属性 | 不要把根的参与状态作全树聚合；见 3.5 节 |
| 异形节点 Config 读取 | 缺失配置时可创建并保存默认配置 | 配置访问并非始终只读；见 3.2 节 |
| ModifyMatter 异常处理 | 清理工艺信息并保存，不统一回滚已写节点 | 无向上传播异常不代表映射完整；见 5.4 节 |
| Mapping 载入 | 已有 TU 的 IsProduct 可被界面设为 true | 标记必须结合入口解释；见 6.5 节 |
| S2 当前对象解析 | 未统一清空上次缓存；未命中不保证变为 null | 应验证与本轮工作消息匹配；见 7.2 节 |
| 异形节点反查 | 按节点对象引用匹配，未命中抛异常 | 同名或同 ID 的新节点不能替代原节点；见 7.6 节 |
| 异形配置焊点清单 | GetAllBondPositionConfigs 最内层加入基岛节点 | 该便捷方法不能视作真实焊点清单；当前目录未见有效调用 |
| 位置辅助服务 | GetSubstratePos 转运动轴，GetModulePos 仍返回 G0 | 不能仅按相近方法名解释参考系；见表 7-4 |
| 清空载具 | 解除区段引用，未统一复位移送状态和动作消息 | 不要当作所有运行状态同时归零；见 6.10 节 |
| 角度示教 | 窗口与调用方分阶段修改不同层级 | 后续取消不表示已写入内容统一撤销；见 8.7 节 |
| 异形基岛数量接口 | GetModuleCount 取各基板下节点数的 Max | 空基板集合会抛异常；非空时返回每板最大数，不是总数 |